

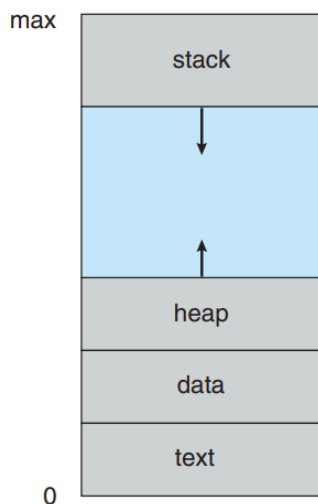
Processi e Thread

Qual è la differenza tra un processo e un thread?

Un processo (o job) è un'istanza di un programma in esecuzione, ed è l'unità a cui il sistema operativo alloca le risorse.

Gli elementi del processo sono: program counter e contenuto dei registri, stack (dati temporanei: variabili locali, indirizzi di return, parametri delle funzioni), la sezione testo (il programma), sezione dati (variabili globali), heap (allocazione dinamica).

Processo in memoria



Un thread è l'unità di esecuzione di un processo a cui il sistema operativo assegna il processore. I thread condividono parte delle loro informazioni con altri thread. Viene detto "processo leggero"

La differenza sostanziale sta nelle risorse e memoria condivise.

Ogni processo ha il proprio spazio di memoria, il che significa che ogni processo è indipendente dagli altri e di conseguenza previene il fatto che altri processi possano corrompere un processo. Per farli comunicare servono meccanismi più complessi come l'IPC (Inter-Process Communication).

I thread di un processo condividono lo stesso spazio di memoria (codice, dati, heap) e le stesse risorse (file aperti). Questo rende la comunicazione tra loro molto semplice e veloce. Pur avendo cose condivise, ogni thread ha un proprio contesto di esecuzione che include: Program Counter, registri e stack. Questo è il TCB (Thread Control Block).

Vantaggi dei thread: creazione/terminazione e switch di contesto sono più veloci tra thread rispetto a processi, grazie al minor overhead.

Nello scheduling, il CPU scheduler opera a livello di thread (non di processo) in un sistema multi thread.

Le risorse (memoria, file) vengono assegnate ai processi o ai thread?

Le risorse vengono assegnate ai processi, che possiedono tutto lo spazio di indirizzamento della memoria e i file aperti.

Cos'è un processo?

Un processo potrebbe essere riassunto nella definizione di "programma in esecuzione". Come è intuibile, di tale programma il sistema operativo deve conoscere diverse informazioni contenute nel PCB.

Tutte queste informazioni sono conservate nella PCB (Process Control Block): una parte della memoria.

Quando avviene un context switch vengono caricate le informazioni del PCB del processo in corso.

Quali sono gli stati di un processo?

New: in fase di creazione

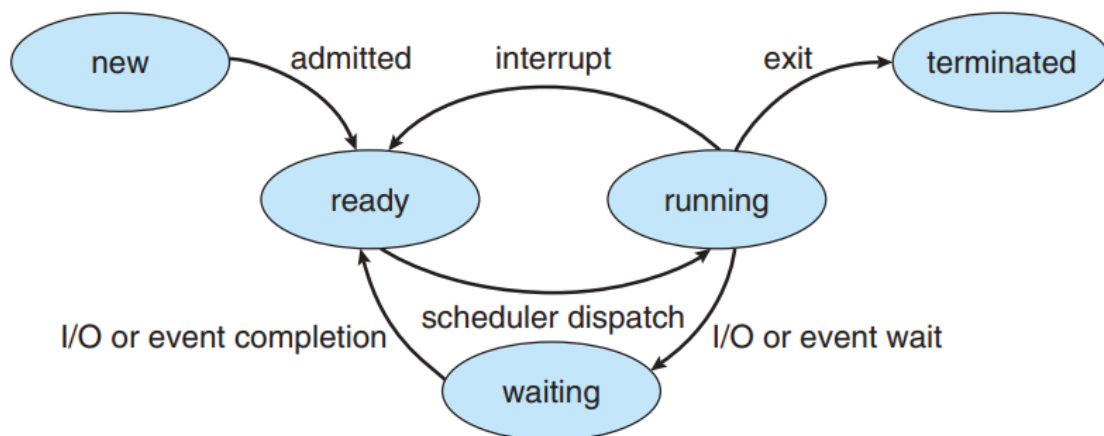
Ready: Il processo attende la disponibilità di un processore.

Running: Il processore sta eseguendo le istruzioni

Waiting: Il processo attende un evento (come il ricevere dati I/O), quindi lascia "di sua volontà" la CPU per permetterne l'utilizzo ad altri processi.

Terminated: il processo ha completato l'esecuzione.

Stati del processo



Descrivi i passaggi di stato: come si passa da Nuovo a Pronto? Da Esecuzione ad Attesa? Da Attesa a Pronto? Da Esecuzione a Pronto?

Da new a ready si passa quando il processo è ammesso (scheduler a lungo termine)

Da ready a running si passa quando lo scheduler decide di eseguire le operazioni del processo (scheduler a breve termine).

Da running a ready quando c'è un rilascio prematuro (causato principalmente dal time slice (quanto di tempo) scaduto in un sistema di scheduling pre rilasciante come il Round Robin).

Da running a waiting quando c'è un'attesa I/O (o una richiesta bloccante).

Da Waiting a ready quando finisce l'attesa I/O.

Da running a terminated quando il processo riceve exit.

Se un processo aspetta delle risorse (es. un'operazione di I/O), in che stato si trova?

Si trova in stato di waiting.

Cos'è il PCB e quali informazioni contiene?

A ogni processo è associata una struttura dati detta descrittore di processo (process control block), contenente tutte le informazioni relative al processo:

- 1) Stato del processo
- 2) Program Counter
- 3) Registri CPU
- 4) Parametri per lo scheduling della CPU
- 5) Informazioni per la gestione della memoria
- 6) Informazioni di accounting
- 7) Informazioni su I/O

Il PCB è la struttura dati che rende possibile il cambio di contesto.

Quando il SO decide di fermare un processo A per eseguirne uno B:

1. Salva lo stato corrente della CPU (PC, registri, etc.) nel PCB di A.
2. Carica lo stato salvato dal PCB di B nei registri della CPU.
3. Il processo B riprende l'esecuzione.

Cos'è lo stato di "zombie"? Quando si verifica?

Quando un processo figlio termina, il padre ha "diritto" di avere accesso al suo PCB (per ricevere dei dati, magari), pertanto per un po' di tempo la PCB dei processi appena terminati rimane in RAM. Una volta che il padre chiama wait(), il PCB viene finalmente deallocato e il processo zombie scompare.

Se il processo padre termina senza aver fatto wait(), i suoi figli zombie diventano orfani, e vengono adottati dal processo init, cioè il processo numero 1.

Scheduling del processore

Abbiamo parlato di PCB, il contenitore delle informazioni di ogni processo. Avendo un sistema multitasking, è necessario poterne immagazzinare diversi: questo avviene nella RAM. Diversi PCB sono immagazzinati nella RAM contemporaneamente, uno soltanto dei quali è assegnato alla CPU.

Sorge pertanto spontanea la domanda: come funziona il cambio di processo? Secondo quali principi? Vediamolo

Quali sono i tipi di scheduling?

Lungo termine (detto job scheduler): seleziona quali processi ammettere nel sistema e li carica in memoria. La sua funzione principale è controllare il grado di multiprogrammazione, ovvero il numero di processi attivi nel sistema.

La scelta del nuovo processo è dettata dal fatto che si tende a creare un equilibrio tra processi CPU-bound (usano molto la CPU) e I/O-bound (usano principalmente I/O).

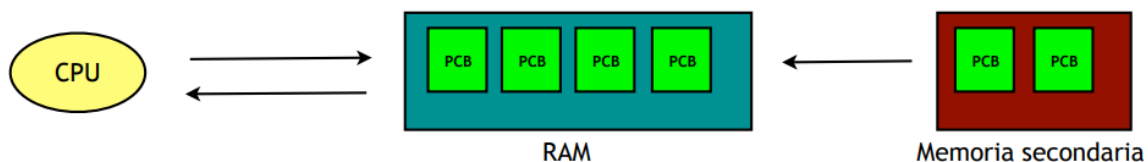
Medio termine (detto swapper)

Lo scheduler a breve termine implementa la funzione di swapping. Non si occupa di far entrare nuovi processi, ma bensì di sospendere temporaneamente processi già esistenti

effettuando uno scambio tra RAM e memoria secondaria per ridurre il grado di programmazione.

Questo significa poter portare avanti più processi contemporaneamente, ma anche rallentare l'esecuzione generale perché un processo in memoria secondaria dovrà prima passare nella RAM per poter accedere alla CPU.

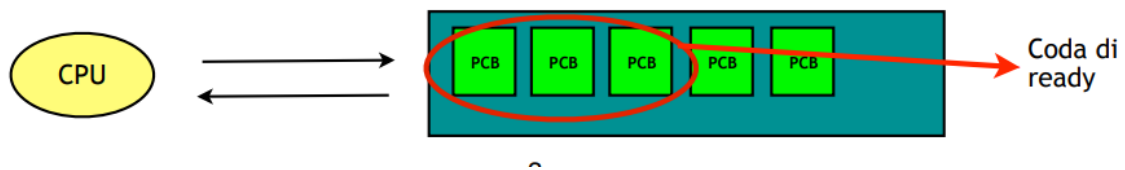
Le decisioni di caricamento/scaricamento sono correlate al controllo del grado di multiprogrammazione e ai requisiti di memoria dei processi (se la memoria è piena o se un processo è inattivo da molto tempo, può essere swappato fuori per fare spazio).



Breve termine (CPU scheduler)

Viene eseguito dal dispatcher.

Pensiamo ad una soluzione totalmente differente: gli unici processi ad aver diritto all'accesso alla CPU sono quelli in ready. Viene quindi generata una "sotto-coda" dentro alla RAM, detta coda di ready.



Viene eseguito con altissima frequenza (ogni pochi millisecondi).

Questa soluzione porta però alla luce molte problematiche: come ad esempio: quale processo merita la CPU? Per quanto tempo? Quando?

Sono necessarie delle politiche di scheduling.

Una volta determinato quale sia il prossimo processo viene interpellato il dispatcher di context switch che assegna il nuovo processo alla CPU.

(il CPU scheduler fa la scelta che poi viene eseguita dal dispatcher).

I criteri delle politiche

Alcuni concetti essenziali.

I criteri sono sia orientati all'utente (relativi alla percezione del comportamento del sistema da parte degli utenti o dei processi, come il tempo di risposta nei sistemi interattivi) e sia orientati al sistema (relativi all'efficienza dell'utilizzazione del processore, come la frequenza del completamento dei processi).

A loro volta, i criteri possono essere relativi alle prestazioni (facilmente misurabili, sono quantitativi) o non relativi alle prestazioni (difficili da misurare, sono qualitativi).

Vediamoli per bene quali sono esattamente i criteri.

Criteri orientati all'utente, relativi alle prestazioni:

tempo di risposta (tempo che trascorre dall'invio di una richiesta fino a quando la risposta non comincia ad essere ricevuto)

tempo di turnaround: intervallo di tempo tra l'invio di un processo e il suo completamento.
scadenze (quando si può specificare il termine di completamento di un processo, lo scheduling può subordinare altri obiettivi).

Un altro criterio relativo all'utente è la prevedibilità: un dato job può essere eseguito nello stesso tempo e allo stesso costo indipendentemente dal carico del sistema.

Criteri relativi al sistema, relativi alle prestazioni:

Throughput (processi completati per unità di tempo, la strategia dovrebbe massimizzare).

Utilizzazione del processore (percentuale del tempo in cui il processore è occupato, molto importante per i sistemi condivisi).

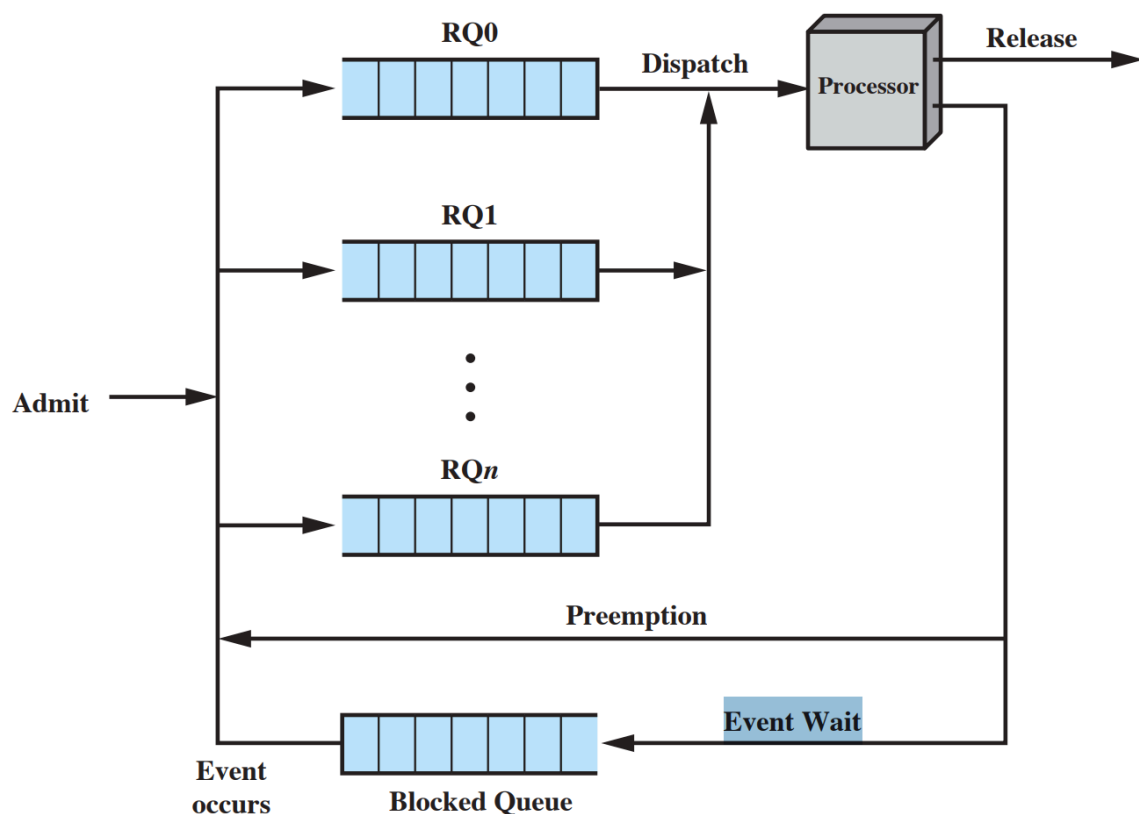
Altri criteri relativi al sistema:

Fairness (in assenza di indicazioni da parte dell'utente o del sistema, i processi devono essere trattati allo stesso modo, senza subire starvation).

Priorità (quando si assegna una priorità ai processi, la strategia di scheduling dovrebbe favorire i processi a più alta priorità).

Bilanciamento delle risorse (la strategia di scheduling favorisce i processi che sotto utilizzano le risorse più impegnate)

Gestione a priorità



I processi sono divisi in ready queue in base alla loro priorità.

Questo schema garantisce che i processi ad alta priorità abbiano accesso preferenziale alla CPU. Se durante l'esecuzione di un processo ce n'è un altro con priorità maggiore, il kernel ne preempt (prerilascia) l'attuale e dispatcha il nuovo arrivato.

Quali sono le politiche (algoritmi) di scheduling del processore più comuni?

Riguardano lo scheduling a breve termine.

Le politiche di scheduling sono semplicemente delle regole che guidano i processi: più precisamente quale tra questi "meriti" la CPU prima degli altri.

L'obiettivo delle politiche è allocare la CPU in modo da ottimizzare alcuni aspetti del comportamento del sistema.

Per farlo, la selezione del processo si basa su: priorità, esigenza di risorse, caratteristiche di esecuzione (tenendo conto dei parametri: w = tempo di attesa, e = tempo di esecuzione, s = tempo stimato di esecuzione).

Tale decisione può avvenire senza prerilascio (il processo continua fino al termine del programma o in un'interruzione I/O) o con prerilascio (il processo in esecuzione viene interrotto e messo in stato ready, avviene per esempio quando arriva un nuovo processo, o periodicamente).

ALGORITMI

FCFS (First Come First Served)

E' la gestione più semplice -> ogni processo pronto è inserito nella coda pronti -> quando un processo termina o è sospeso, viene selezionato il processo in ordine di arrivo. Favorisce processi lunghi rispetto a quelli brevi. Favorisce processi CPU-bound rispetto a quelli I/O bound.

Round robin

Questa politica è usata spesso in sistemi time-sharing (con più utenti da servire).

Sostanzialmente la coda è organizzata come la politica FCFS, ma viene aggiunta una nuova particolarità: il quanto di tempo (q , il quantum).

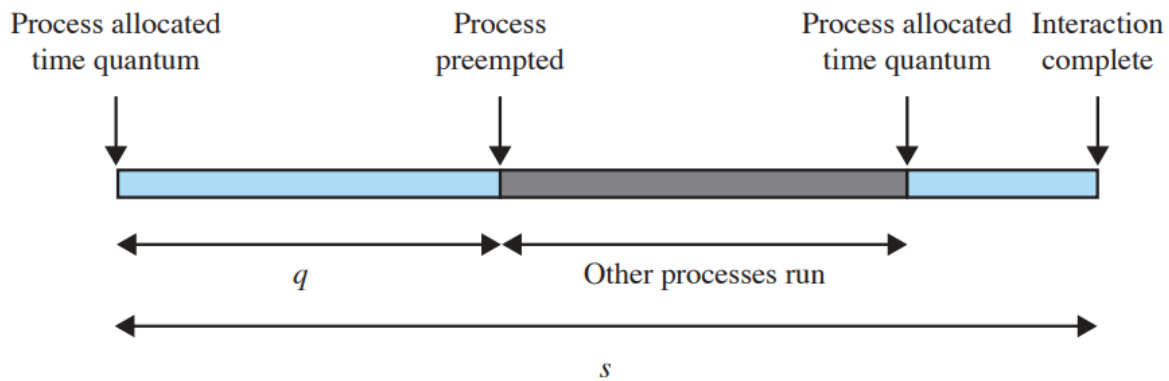
Il quanto di tempo è una costante che viene impostata dal sistema operativo, la quale ha il ruolo di riassegnare la CPU una volta passate x unità di tempo.

Viene da se che quindi utilizza il prerilascio, basato su clock (time slicing).

Si crea pertanto una rotazione (ecco il motivo del nome).

La scelta del quanto di tempo chiaramente è discriminatoria... con un quanto troppo grosso, il sistema Round Robin sostanzialmente non esiste e diventa un semplice FCFS. Con un quanto troppo piccolo invece il numero di context switch aumenterebbe incredibilmente, rallentando il sistema (overhead).

(nota: overhead è il costo aggiuntivo in termini di risorse che non viene utilizzato direttamente per l'esecuzione delle applicazioni utente, ma serve a far funzionare il sistema stesso)



SPN (Shortest Process Next)

Seleziona il processo col minimo tempo stimato di esecuzione.

Non utilizza prerilascio.

Ordinare i processi nella coda in base al tempo di esecuzione (dal più veloce al più lento, dove un processo breve può saltare in testa alla coda pronti) riduce il tempo medio di attesa della coda.

E' possibile la starvation per processi lunghi ed è inadatto nei sistemi time-sharing data l'assenza di prerilascio.

Ma come calcolare il tempo di esecuzione di un processo (CPU-Burst)?

Viene definito tramite delle operazioni "statistiche" che fa il sistema operativo, come la media aritmetica della durata delle esecuzioni precedenti o la media esponenziale

Media aritmetica della durata delle esecuzioni precedenti

$$S_{n+1} = \frac{1}{n} \sum_{i=1}^n T_i = \frac{T_n + (n-1)S_{n-1}}{n}$$

Media esponenziale

$$S_{n+1} = \alpha \sum_{i=1}^n (1-\alpha)^{n-i} T_i + (1-\alpha)^n S_1 = \alpha T_n + (1-\alpha)S_n$$

$$0 < \alpha < 1.$$

La media esponenziale è più usata (più leggera da calcolare e garantisce previsioni migliori).

In sostanza, i termini iniziali (più vecchi) perdono di significato man mano che si va avanti, mentre invece i termini più recenti sono quelli che valgono più.

SRT (Shortest Remaining Time)

Seleziona il processo col minimo tempo stimato di esecuzione.

Utilizza prerilascio (questa è la differenza principale con SPN).

Se arriva nella coda un nuovo processo con tempo residuo rimasto stimato minore del processo in esecuzione, allora lo scheduler sospende il job corrente e assegna la CPU al nuovo arrivato.

HRRN (Highest Response Ratio Next)

Seleziona il processo con il più alto rapporto di risposta (tempo di turnaround normalizzato).

Non ha prerilascio.

$$R = \frac{w + s}{s}$$

ove w = tempo di attesa e s = tempo stimato di esecuzione.

Ha un aging intrinseco: più un processo resta in attesa, più cresce w , e dunque R , prevenendo la starvation dei job "lunghi".

Favorisce i processi brevi ma bilancia le preferenze con la necessità di servire anche quelli più lunghi, senza ricorrere al prerilascio.

Schedulazione fair-share e schedulazione UNIX tradizionale

Fair-share:

Le politiche di schedulazione vengono basate su gruppi di processi, ognuno dei quali ha una "frazione" del processore. Ha l'obiettivo di monitorare l'uso per dare meno risorse a chi ne ha avuto più della frazione spettante.

Nella schedulazione fair-share, la priorità di un processo non è statica, ma viene calcolata dinamicamente.

Si può assegnare una priorità determinata con

$$CPU_j(i) = \frac{CPU_j(i-1)}{2}, \quad GCPU_k(i) = \frac{GCPU_k(i-1)}{2},$$
$$P_j(i) = Base_j + \frac{CPU_j(i)}{2} + \frac{GCPU_k(i)}{4 \cdot W_k}$$

UNIX tradizionale

Utilizza code di priorità con round-robin in ciascuna coda e prerilascio ogni 1s.

La schedulazione UNIX tradizionale usa un approccio a feedback multilivello per favorire i processi interattivi a scapito di quelli CPU-bound. Un processo che usa la CPU per brevi periodi (come un I/O), rimane nelle code a priorità alta. Un processo che usa sempre tutto il suo quanto di tempo (quindi CPU-bound), viene spostato verso code a priorità bassa.

Questo meccanismo premia i processi che rilasciano spesso la CPU, per questo è efficiente per i sistemi time-sharing.

Cosa si intende per "prerilascio" (preemption) di un processo?

Il prerilascio è la possibilità che il sistema operativo interrompa un processo in esecuzione e lo rimetta in stato di ready. Può avvenire in 3 casi:

- quando arriva un processo con priorità più alta

- quando un'interruzione I/O riattiva un processo waiting (per esempio abbiamo appena ricevuto l'input, il processo da waiting torna ready, allora potrebbe avere una priorità più alta di quello in esecuzione,
- periodicamente, in base a un clock (come lo scadere del quanto di tempo, succede nell'algoritmo Round Robin)

Quali sono gli obiettivi generali delle politiche di scheduling? (generata)

Gli obiettivi di una politica di scheduling sono i criteri di performance che si cerca di ottimizzare.

Possiamo suddividerli in obiettivi orientati all'utente e obiettivi orientati al sistema.

Obiettivi orientati all'utente:

- Minimizzare il tempo di risposta: cruciale nei sistemi interattivi, è il tempo che passa da una richiesta all'inizio di una risposta.
- Minimizzare il tempo di turnaround: importante per i job batch, è il tempo totale che un processo passa nel sistema.

Obiettivi orientati al sistema:

- Massimizzare il throughput: completare il maggior numero di processi possibile nell'unità di tempo.
- Massimizzare l'utilizzazione della CPU: tenere il processore occupato il più a lungo possibile.
- Garantire fairness (equità): assicurare che ogni processo riceva una giusta quota di tempo CPU, evitando la starvation.

È importante notare che questi obiettivi sono spesso in conflitto tra loro. Ad esempio, minimizzare il tempo di risposta per i processi interattivi potrebbe ridurre il throughput generale, perché richiede frequenti cambi di contesto. Il compito di una buona politica di scheduling è trovare un compromesso adeguato al tipo di sistema (es. batch, interattivo, real-time).

Quale politica massimizza il numero di lavori portati a termine (throughput)?

Shortest Process Next e Shortest Remaining Time (entrambi servono prima i processi più brevi).

In teoria, SRT può offrire un throughput leggermente superiore perché è più reattivo nell'eseguire immediatamente i processi molto brevi appena arrivano. Tuttavia, questo vantaggio può essere annullato dall'aumento dell'overhead dovuto ai maggiori cambi di contesto che il prerilascio comporta

Concorrenza e mutua esclusione

Race condition

Si verifica quando più processi o threads scrivono dati in modo che il risultato finale dipende dall'ordine di esecuzioni delle istruzioni dei processi.

Mutex

Primitiva di sincronizzazione utilizzabile per assicurare la mutua esclusione.

Grado di consapevolezza fra processi

- Processi ignari l'uno dell'altro: gareggiano solo per accedere a risorse (CPU, I/O)
- Processi che condividono memoria: cooperano indirettamente ma devono proteggere la sezione critica
- Processi che comunicano: si coordinano esplicitamente ma possono comunque finire in deadlock e in starvation.

Cosa si intende per mutua esclusione?

Per mutua esclusione si intende che un solo processo alla volta è ammesso alla sezione critica per ciascuna risorsa condivisa.

I requisiti fondamentali:

- Un solo processo nella sezione critica
- Solo i processi che NON sono nella sezione rimanente possono partecipare alla decisione per entrare nella sezione critica (progress) **DA MIGLIORARE**
- Evitare deadlock: se nessun processo è nella sezione critica e più processi vogliono accedervi, almeno uno deve entrare senza posticipare la decisione.
- Evitare starvation: prima o poi il processo deve poter accedere alla sezione critica (BOUNDED WAITING). C
- Se nessun processo è nella sezione critica, deve essere concesso a ogni processo di entrare senza ritardo.
- Un processo resta nella sezione critica per un tempo finito
- La soluzione non deve fare assunzioni sulla velocità dei processi o sul numero di CPU disponibili.

Come si può realizzare la mutua esclusione? (soluzioni software, soluzioni hardware)

La mutua esclusione è realizzabile in vari modi.

- Ci sono soluzioni hardware. Su un sistema singolo processore (uniprocessore), si può garantire la mutua esclusione disabilitando le interruzioni prima di entrare nella sezione critica e riabilitandole subito dopo. Questo impedisce al sistema operativo di fare cambi di contesto, ma rallenta il sistema e non è applicabile su multiprocessore. Oppure, a volte, l'hardware può fornire istruzioni di lettura/scrittura atomiche. Questo è applicabile sia su uniprocessore che multiprocessore, ma porta ad attesa attiva e possibilità di stallo e starvation.
- Ci sono soluzioni puramente software che comprendono l'utilizzo di algoritmi come l'algoritmo di Dekker e l'algoritmo di Peterson.
- Ci sono soluzioni ad alto livello fornite dal Sistema Operativo (o da librerie) come semafori e monitor

Descrivi gli algoritmi di Peterson e Dekker

Algoritmo di Dekker

E' un protocollo che permette a due processi di assicurare la mutua esclusione

- no supporto hardware, no deadlock, no starvation

Prima di arrivare alla soluzione finale, passiamo per 4 tentativi.

Primo tentativo

```

/* PROCESS 0 */
...
while (turn != 0)
    /* do nothing */ ;
/* critical section*/;
turn = 1;
...

```

```

/* PROCESS 1 */
...
while (turn != 1)
    /* do nothing */ ;
/* critical section*/;
turn = 0;
...

```

Si usa solo la variabile turn per alternare l'accesso. Sembrerebbe funzionare, ma il problema è che se un processo non dovesse mai arrivare a scrivere turn (perchè magari va in waiting), l'altro processo rimarrebbe bloccato.

Secondo tentativo

```

/* PROCESS 0 */
...
while (flag[1])
    /* do nothing */;
flag[0] = true;
/*critical section*/;
flag[0] = false;
...

```

```

/* PROCESS 1 */
...
while (flag[0])
    /* do nothing */;
flag[1] = true;
/*critical section*/;
flag[1] = false;
...

```

Si usano due flag che dovrebbero segnalare che la sezione critica sta venendo eseguita da uno dei due processi, questo non funziona poiché se i due processi arrivano quasi contemporaneamente con entrambe le flag false, entrano entrambi nella sezione critica poiché pensano che essa sia libera.

Terzo tentativo

```

/* PROCESS 0 */
...
flag[0] = true;
while (flag[1])
    /* do nothing */;
/* critical section*/;
flag[0] = false;
...

```

```

/* PROCESS 1 */
...
flag[1] = true;
while (flag[0])
    /* do nothing */;
/* critical section*/;
flag[1] = false;
...

```

Scambiamo l'ordine delle operazioni di scrittura e lettura, prima occupo, poi aspetto che l'altra sia liberata, ma anche qui se i due processi partissero quasi insieme le due variabili

sarebbero inizializzate entrambe a true, così entrambi i loop while sarebbero infiniti e i due processi non avanzeranno mai, è un esempio palese di stallo.

Quarto tentativo

```
/* PROCESS 0 */
...
flag[0] = true;
while (flag[1]) {
    flag[0] = false;
    /*delay */;
    flag[0] = true;
}
/*critical section*/;
flag[0] = false;
...

/* PROCESS 1 */
...
flag[1] = true;
while (flag[0]) {
    flag[1] = false;
    /*delay */;
    flag[1] = true;
}
/*critical section*/;
flag[1] = false;
...
```

Se un processo rileva che un altro processo ha flag true blocca se stesso, poi fa un delay che permette all'altro processo di avanzare e di entrare nella sezione critica per poi impostare flag a false.

Soluzione finale

```
boolean flag [2];
int turn;

void P0()
{
    while (true) {
        flag [0] = true;
        while (flag [1]) {
            if (turn == 1) {
                flag [0] = false;
                while (turn == 1) /* do nothing */;
                flag [0] = true;
            }
        }
        /* critical section */;
        turn = 1;
        flag [0] = false;
        /* remainder */;
    }
}

void P1()
{
    ...
}
```

Come funziona?

Ogni processo segnala la propria intenzione ($\text{flag}[i] = \text{true}$) prima di attendere, e utilizza turn come arbitro: se l'altro è in sezione critica, chi non ha il permesso turn si ritira e attende che il permesso venga lasciato.

Con questa soluzione però paghiamo l'attesa attiva che è un costo non indifferente (perché si può entrare in un loop continuo controllando $\text{flag}[1]$ e turn).

Live scenario:

P0 setta $\text{flag}[0]$ a true

P1 setta $\text{flag}[1]$ a true

(entrambi vogliono entrare nella sezione critica)

turn è ancora a 0.

P0 va nel $\text{while } \text{flag}[1]$ che è vero, quindi passa a $\text{if } (\text{turn} == 1)$, ma questa è falsa.

P0 rimane bloccato nel $\text{while } \text{flag}[1]$, aspettando P1 che setti $\text{flag}[1]$ a false.

P1 va nel $\text{while } \text{flag}[0]$, che è vero. Poi vede $\text{if } \text{turn} == 0$, che è vero, quindi prosegue nell' if .

Setta $\text{flag}[1] = \text{false}$ passando quindi il turno a P0 (è gentile). P1 entra nel ciclo $\text{while } \text{turn} == 0$, aspettando che P0, una volta uscito dalla sezione critica, setti il turn a 1.

P0 ora esce dal $\text{while } \text{flag}[1]$ e può eseguire la sua sezione critica.

P0 setta $\text{turn} = 1$, $\text{flag}[0] = \text{false}$.

P1 ora esce dal $\text{while } \text{turn} == 0$, resetta $\text{flag}[1] = \text{true}$.

P1 può uscire dal while ed eseguire la sezione critica

Sostanzialmente, chi non ha il turn si sacrifica e mette la propria flag a false, aspetta il proprio turn e rimette la flag a true.

Algoritmo di Peterson

L'algoritmo di Dekker si può scrivere in maniera più elegante con l'algoritmo di Peterson.

Funzionamento:

abbiamo bisogno di 2 variabili condivise fra i 2 processi.

boolean $\text{flag}[2]$ -> per indicare se un processo vuole entrare nella sezione critica

int turn -> indica di chi è il turno per entrare nella sezione critica

Quando un processo vuole entrare nella sezione critica, setta la sua flag a true.

Poi setta la turn con il valore dell'altro processo (es: 1).

È "humble" (per modo di dire), nel senso che quando un processo vuole entrare, setta il turn all'altro processo (come quando entriamo nel pullman in 2 persone e lasciamo entrare prima l'altro). Quindi se il processo 1 vuole entrare, fallo entrare prima di 0.

Poi si arriva al while . Questo non fa assolutamente nulla oltre a continuare a controllare la condizione. Si rimane nel while finché la flag e il turno sono dell'altro processo.

Quando si supera il while , è possibile entrare nella sezione critica.

Dopo sezione critica, si setta la propria flag a falso.

L'algoritmo rispetta tutte le regole della sezione critica, ma ha comunque i suoi limiti: è previsto solamente per 2 processi, ed inoltre prevede una attesa attiva (cioè l'altro processo cicla "a vuoto" mentre aspetta, si potrebbe investire meglio quel tempo!)

```

boolean flag [2];
int turn;

void P0()
{
    while (true) {
        flag [0] = true;
        turn = 1;
        while (flag [1] && turn == 1) /* do nothing */;
        /* critical section */;
        flag [0] = false;
        /* remainder */;
    }
}

void P1() { . . . }

```

il codice per i 2 processi è uguale tranne per la condizione del while, dove si mette 0 al posto di uno per indicare l'altro processo: processo 0 mette 1, processo 1 mette 0.

Live scenario:

abbiamo 2 processi, 0 e 1, che voglio entrare nella sezione critica.

P0 setta flag[0] = true

P1 setta flag[1] = true

P0 setta turn = 1

P1 setta turn = 0

turn (come flag) è una variabile condivisa, quindi alla fine turn vale 0, settata da P1.

ora quale while dei 2 processi va avanti e quale rimane la?

P0

while (flag[1] && turn == 1);

P1

while (flag[0] && turn == 0);

il while di P0 è falso, quindi può passare alla sezione critica.

il while di P1 è vera, quindi rimane la.

P0, dopo la sezione critica, setta flag[0] = false

Ora il while di P1 è falso e può passare alla sezione critica e poi setta flag[1] a false.

Cos'è un semaforo?

Tecnica sviluppata da Dijkstra.

Risolvono il problema della sincronizzazione.

Sono gestiti dal sistema operativo e sono formati da:

- una variabile con valore intero (nel semaforo generico può essere negativa)
- una coda
- 3 operazioni

Le operazioni (atomiche) sono:

1. L'inizializzazione della variabile a un valore non negativo
2. semWait -> decrementa il valore del semaforo. Se il nuovo valore è negativo, il processo è sospeso e inserito nella coda.
3. semSignal -> incrementa il valore del semaforo. Se il nuovo valore è minore o uguale a 0 (quindi non diventa positivo nonostante l'incremento), uno dei processi della coda è riattivato.

Osservazione:

Il processo non può sapere in anticipo se con semWait verrà sospeso, se c'è già qualche processo che attende semSignal, o quale processo verrà riattivato con semSignal.

```
struct semaphore {  
    int count;  
    queueType queue;  
};
```

Semaforo generico

```
void semWait(semaphore s)  
{  
    s.count--;  
    if (s.count < 0) {  
        /* place this process in s.queue */;  
        /* block this process */;  
    }  
}
```

```
void semSignal(semaphore s)  
{  
    s.count++;  
    if (s.count <= 0) {  
        /* remove a process P from s.queue */;  
        /* place process P on ready list */;  
    }  
}
```

I semafori possono essere anche binari, dove semplicemente il valore che può assumere la variabile è 0 o 1. Le operazioni sono le stesse, ma cambia leggermente il comportamento.

- L'inizializzazione è a 0 o a 1.
- semWaitB prima verifica il valore. Lo decrementa solo se non era 0.
- semSignalB prima verifica la coda. Lo incrementa solo se non ci sono processi nella coda.

Chiaramente, le operazioni semWait e semSignal sono devono essere primitive atomiche, tramite soluzioni software, hardware o con l'algoritmo di Peterson.

Cosa sono i messaggi?

Lo scambio di messaggi tra processi permette di realizzare sincronizzazione (per la mutua esclusione) e comunicazione (per lo scambio di informazioni).

Ci sono 2 operazioni primitive:

- send(destination, message): inviare informazioni ad un altro processo
- receive(source, message): riceve informazioni e le salva in un indirizzo.

Sia send che receive possono essere bloccanti o non bloccanti.

Quando il destinatario esegue la receive, se è già presente un messaggio, lo riceve e continua l'esecuzione. Se non è presente un messaggio, può decidere se attendere o proseguire l'esecuzione senza più ricevere il messaggio.

Si può scegliere una combinazione come:

- send bloccante e receive bloccante (permette la sincronizzazione dei processi).
Detto Rendez-vous.
- send non bloccante e receive bloccante (se il mittente deve mandare più messaggi a tanti destinatari, come un processo server)
- send non bloccante e receive non bloccante (nessuna attesa)

L'indirizzamento può essere diretto o indiretto

- diretto -> Il mittente specifica il processo del destinatario. La ricezione può essere implicita se il destinatario sa già da chi aspettarsi il messaggio. Esplicita se receive lo ottiene comunque senza doverlo preconsocere (in pratica si aspetta il messaggio da chiunque).
- indiretto -> i messaggi non vanno ad un destinatario specifico ma vanno verso una struttura dati condivisa (mailbox). Il destinatario preleva il messaggio da questa.

I messaggi hanno un formato (header, body) e vengono gestiti in code tipicamente FIFO.

Cos'è il problema del produttore e consumatore?

Anche questi, come i semafori, risolvono il problema della sincronizzazione (della memoria condivisa)).

Abbiamo un buffer, dove i produttori devono depositare il loro prodotto (dati), e dove i consumatori devono prelevare i prodotti già presenti nel buffer. Ovviamente le 2 azioni devono essere atomiche (un solo produttore o consumatore alla volta accede al buffer).

Bisogna assicurare quindi la mutua esclusione, evitare che un produttore aggiunga dati ad un buffer pieno, evitare che il consumatore cerchi di estrarre dati da un buffer vuoto.

Questa è la soluzione al problema che utilizza semafori generici.

Utilizza un semaforo binario (s), un semaforo contatore (n) per il numero di elementi pieni nel buffer, un semaforo contatore (e) che rappresenta il numero di spazi vuoti nel buffer


```

/* program boundedbuffer */
const int sizeofbuffer = /* buffer size */;
semaphore s = 1, n= 0, e= sizeofbuffer;

void producer()
{
    while (true) {
        produce();
        semWait(e);
        semWait(s);
        append();
        semSignal(s);
        semSignal(n);
    }
}

void consumer()
{
    while (true) {
        semWait(n);
        semWait(s);
        take();
        semSignal(s);
        semSignal(e);
        consume();
    }
}

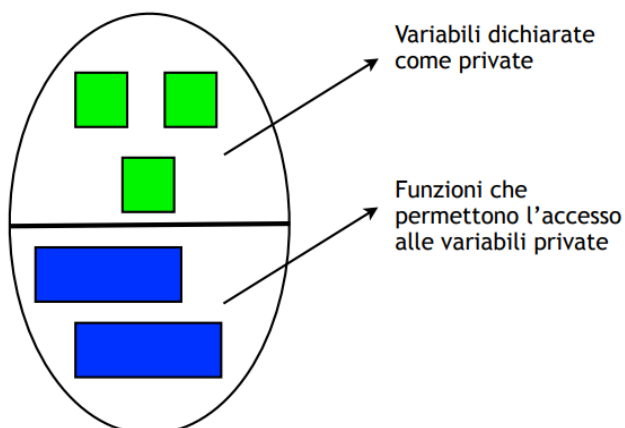
```

Il produttore prima produce l'elemento, poi attende che ci siano spazi liberi (semafore e), poi attendo di poterlo effettivamente inserire (semaforo s, magari un consumatore o un altro produttore stavano scrivendo), inserisce il dato, si libera dall'occupazione del buffer, incrementa la quantità di posizioni occupate (semaforo n)

Il consumatore prima verifica che ci siano elementi da consumare, poi attende di poter accedere al buffer, estrae il dato, rilascia l'accesso al buffer, segnala che c'è uno spazio vuoto in più e infine consuma l'elemento.

Cos'è un monitor e perché è stato introdotto?

Il monitor è un costrutto implementato in certi linguaggi di programmazione che permette la gestione in mutua esclusione di alcune variabili. E' il monitor stesso che garantisce la mutua esclusione. Questo sistema permette, dato un accesso casuale a più processi alla risorsa, un accesso ordinato.



Il modulo software è costituito da: codice di inizializzazione (per i processi che vogliono usare il monitor), dati locali, procedure.

I dati locali sono accessibili solo all'interno del monitor attraverso le procedure.

I processi accedono invocando una delle procedure.

Un solo processo alla volta può essere in esecuzione nel monitor. Ma come?

Si utilizzano delle variabili, che hanno tipo "condition". La variabile viene dichiarata così: condition c;

e possiede 2 funzioni per essere gestita:

- cwait(c) -> sospende il processo che esegue questa operazione finché un altro processo non invoca csignal(c).
- csignal(c) -> causa il risveglio di un processo sospeso, se non ce ne sono non fa nulla.

Se un processo che utilizza il monitor vuole modificare dei dati condivisi ma sono usati da un altro processo, c'è un'operazione di wait. Quando il processo che utilizza i dati condivisi finisce, manda il signal e l'altro processo riprende l'esecuzione.

Quale problema dei semafori risolve?

Offre una soluzione più modulare, che sfrutta l'incapsulamento ed è meno soggetta ad errori. Risolvono anche i deadlocks.

Fornisco un alto livello di astrazione

Può essere usato per risolvere il problema produttori/consumatori.

Problema dei lettori e scrittori

E' simile al problema dei produttori e dei consumatori. Qua però alcuni processi leggono e basta (lettori) ed altri scrivono soltanto (scrittori).

Più lettori possono leggere simultaneamente i dati, ed un solo scrittore alla volta li può modificare.

Ci sono 2 soluzioni: quella che dà precedenza ai lettori e quella che dà precedenza ai scrittori.

Stallo (deadlock)

Che cos'è uno stallo (deadlock)?

Un insieme di processi è in stallo (deadlock) se ogni processo è in attesa di un evento che può essere generato solo da un altro processo nell'insieme.

- è permanente
- nel caso generale, non ha nessuna soluzione efficiente
-

Condizioni per lo stallo

Condizioni necessarie:

- mutua esclusione (caratteristico della risorsa)
- possesso e attesa (facoltà di appropriarsi di una risorsa e chiederne un'altra)
- assenza di prerilascio

Condizione determinante:

- attesa circolare

Azioni contro lo stallo

- Potremmo **prevenirlo** progettando il sistema correttamente. Si possono utilizzare metodi indiretti o metodi diretti.

Indiretti -> prevengono il presentarsi di una delle condizioni necessarie: la mutua esclusione è evitabile solo in rari casi, possesso e attesa si potrebbe imporre un'unica richiesta globale con tutte le risorse ai processi da fare all'inizio, assenza di preilascio potremmo, nel momento in cui un processo va in attesa, fargli rilasciare tutte le risorse che possiede.

Diretti -> prevengono il presentarsi dell'attesa circolare. Per farlo, le risorse vengono divise in categorie ordinate per numero ed un processo può richiedere solo risorse di categorie successive rispetto a quelle già assegnate.

- Questi metodi funzionano, ma hanno molti punti deboli e rallentamenti.

Potremmo **evitarli (prima che accadano, sono previsioni!)**.

Come?

Rifiutando l'esecuzione per i processi le cui richieste potrebbero determinare stallo

Rifiutando l'allocazione per le richieste di risorse che potrebbero determinare lo stallo

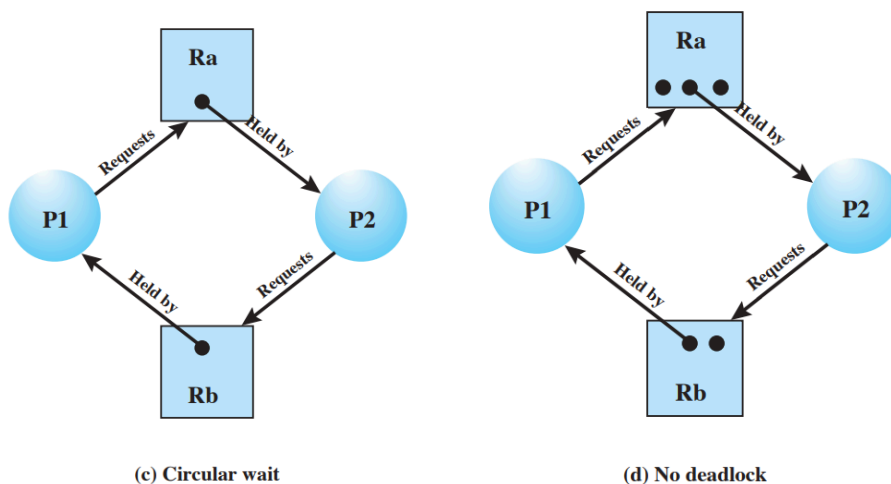
In entrambi i casi, è necessario conoscere le future richieste dei processi.

Si usa l'algoritmo del banchiere.

- Infine si può rilevare lo stallo se si verifica l'attesa circolare tramite un algoritmo eseguito periodicamente. A differenza dell'algoritmo del banchiere, non guarda alle necessità future, ma solo alle richieste attuali dei processi. Prova a simulare la terminazione dei processi, i processi che non possono essere soddisfatti sono in stallo.

Lo svantaggio è l'overhead delle verifiche. Chiaramente, una volta rilevato lo stallo, serve una strategia per risolverlo. Come? Potremmo terminare tutti i processi in stallo (anche uno alla volta), o revocare le risorse al processo per darle a un altro.

In cosa consiste la condizione di "attesa circolare"?



L'attesa circolare è una condizione determinante per il deadlock dove il processo P1 richiede una risorsa che è posseduta da P2 ed allo stesso tempo P2 richiede una risorsa posseduta da P1.

Categorie di risorse

- risorse riutilizzabili: sono utilizzabili da un solo processo alla volta, non vengono distrutte dal loro utilizzo (processore, canali I/O, memoria, files, semafori...)
- risorse consumabili; possono essere create e distrutte (interruzioni, segnali, messaggi, dati in buffer I/O)

Algoritmo del banchiere

Il sistema operativo deve monitorare ciò che fanno i processi, d'altro canto loro gli devono fornire delle informazioni sulla loro esecuzione. In particolare, lo stato delle risorse (cioè quante risorse stanno usando e quante ne vogliono ancora usare).

Uno stato sicuro è una situazione in cui esiste almeno una sequenza di allocazioni che porta a termine tutti i processi.

L'algoritmo del banchiere permette di evitare lo stallo rifiutando una se porta in uno stato insicuro (che finirebbe in deadlock).

Il sistema ha 4 strutture

I vettori R e V e le matrici C e A.

R1	R2	R3
9	3	6

Resource vector **R**

R1	R2	R3
0	1	1

Available vector **V**

	R1	R2	R3
P1	3	2	2
P2	6	1	3
P3	3	1	4
P4	4	2	2

Claim matrix **C**

	R1	R2	R3
P1	1	0	0
P2	6	1	2
P3	2	1	1
P4	0	0	2

Allocation matrix **A**

Vettore R: totale di ogni tipo di risorsa nel sistema

Vettore V: vettore delle risorse attualmente libere

Matrice C: per ciascun processo P, la quantità massima ogni risorsa R che potrebbe chiedere

Matrice A: per ciascun processo P, la quantità di ogni risorsa R già allocata

Facendo $C - A$, si ottiene una matrice con le risorse che potrebbero essere ancora richieste da ogni processo.

Esempio di allocazione rifiutata

	R1	R2	R3		R1	R2	R3		R1	R2	R3
P1	3	2	2	P1	1	0	0	P1	2	2	2
P2	6	1	3	P2	5	1	1	P2	1	0	2
P3	3	1	4	P3	2	1	1	P3	1	0	3
P4	4	2	2	P4	0	0	2	P4	4	2	0
Claim matrix C				Allocation matrix A				C - A			
	R1	R2	R3		R1	R2	R3				
	9	3	6		1	1	2				
Resource vector R				Available vector V							

(a) Initial state

	R1	R2	R3		R1	R2	R3		R1	R2	R3
P1	3	2	2	P1	2	0	1	P1	1	2	1
P2	6	1	3	P2	5	1	1	P2	1	0	2
P3	3	1	4	P3	2	1	1	P3	1	0	3
P4	4	2	2	P4	0	0	2	P4	4	2	0
Claim matrix C				Allocation matrix A				C - A			
	R1	R2	R3		R1	R2	R3				
	9	3	6		0	1	1				
Resource vector R				Available vector V							

(b) P1 requests one unit each of R1 and R3

Chiaramente se P1 chiede una unità di R1 e R3. Ma R1 non è disponibile, quindi allocazione rifiutata.

L'algoritmo del banchiere entra in gioco quando le risorse **sono disponibili**, ma allocarle potrebbe essere pericoloso. Funzionamento:

Validità richiesta

Arriva una richiesta da P_i per un certo numero di risorse.

La richiesta fallisce se chiedere risorse che non ci sono oppure se richiede più risorse di quanto aveva dichiarato ($Richesta \leq C[i] - A[i]$? NO, falso)

Simulazione

Se i primi controlli passano, il sistema NON alloca subito le risorse. Simula invece un'esecuzione della richiesta.

$disponibili[i] = disponibili[i] - Richiesta$ (diminuisco le risorse disponibili). Disponibili è V
 $assegnate[i] = assegnate[i] + Richiesta$ (aumento le risorse assegnate a P). Assegnate è A
 A questo punto, esegue l'algoritmo di rilevazione dello stato sicuro su questo stato ipotetico appena simulato.

Decisione finale

Se lo stato ipotetico è **sicuro**: perfetto, significa che dopo questa allocazione esiste ancora un modo per far terminare tutti i processi. La richiesta è accettata

Se lo stato ipotetico è **insicuro**: il sistema ripristina lo stato originale (butta via la simulazione) ed il processo viene sospeso. Chiaramente la richiesta è rifiutata, il processo deve aspettare che altri processi rilascino risorse.

Algoritmo di rilevazione (pseudocodice)

```
1  W ← V
2  while c'è un processo  $P_i$  non spuntato, tale che  $C_{ij} - A_{ij} \leq W_j$  per ogni  $j$ 
3      spunta  $P_j$ 
4      for all  $j$ 
5           $W_j \leftarrow W_j + A_{ij}$ 
6  if tutti i processi sono spuntati
7      return SICURO
8  else
9      return INSICURO
```

Serve a vedere se c'è una sequenza di allocazioni per far terminare tutti i processi senza stallo. Funzionamento:

Per ogni processo, lo si prova a spuntare.

Un processo viene spuntato se le risorse che potrebbe ancora chiedere sono minori o uguali a quelle disponibili (quindi può essere completato con le risorse disponibili). Una volta spuntato (e quindi avrebbe terminato), rilasciamo le sue risorse ($W = W + A[i]$) così che saranno disponibili per gli altri.

Se alla fine i processi sono tutti spuntati, significa che effettivamente esiste una sequenza sicura e che lo stato che abbiamo simulato prima è sicuro.

Gestione della memoria

In che modo i processi vengono caricati in memoria principale?

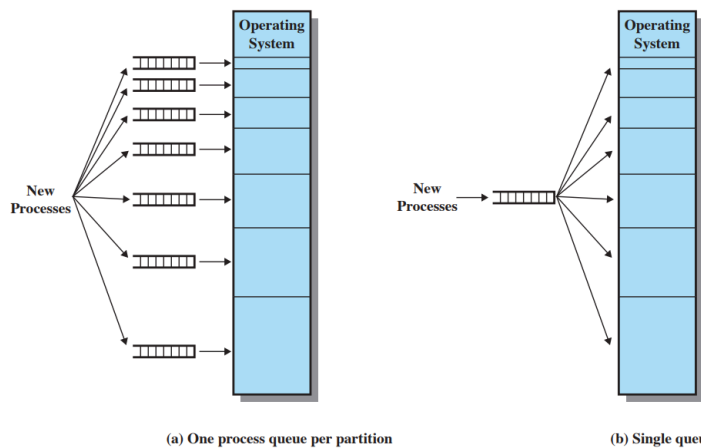
Il gestore della memoria carica i processi in memoria virtuale per essere eseguiti. Si basa su 3 tecniche: partizionamento, segmentazione e paginazione.

- Partizionamento: obsoleto, non prevede l'uso di memoria virtuale. Può usare partizioni fisse o variabili.

Nelle partizioni fisse la RAM è divisa in blocchi di dimensione prefissata.

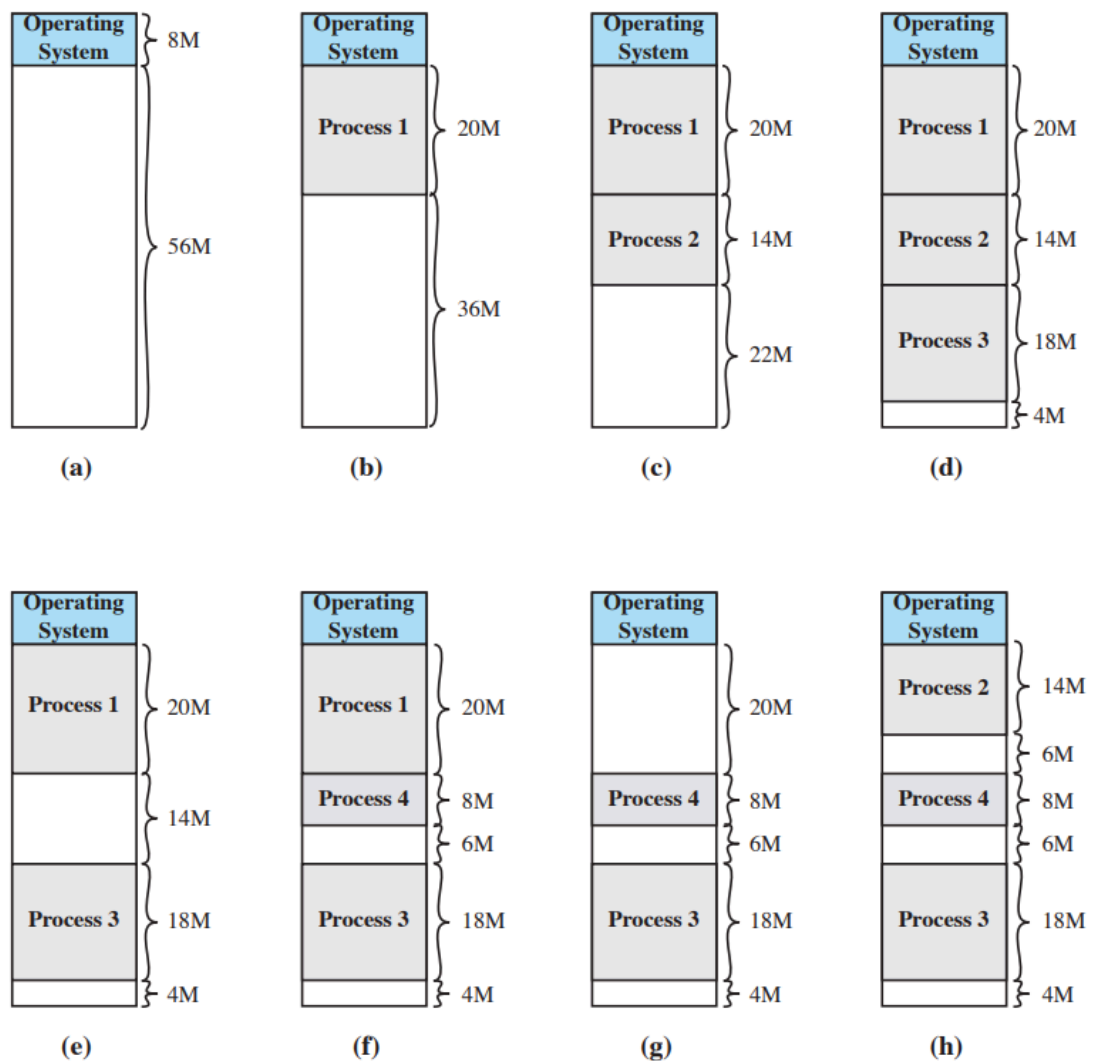
Questa ha vari svantaggi perché un programma potrebbe essere troppo grande per entrare in una partizione (necessità dell'overlay), poi limita il numero di processi attivi, utilizzo inefficiente della memoria perché molto spazio resta inutilizzato quando un programma non occupa tutta la partizione (frammentazione interna, dentro la part)

Algoritmo di allocazione



Partizione dinamiche. Hanno dimensione variabili e sono create dinamicamente quando un processo è caricato in memoria.

Lo svantaggio è la frammentazione esterna (cioè esterna alla partizione), risolvibile con la compattazione.



Qua ci sono 3 tipi di algoritmi di allocazione.

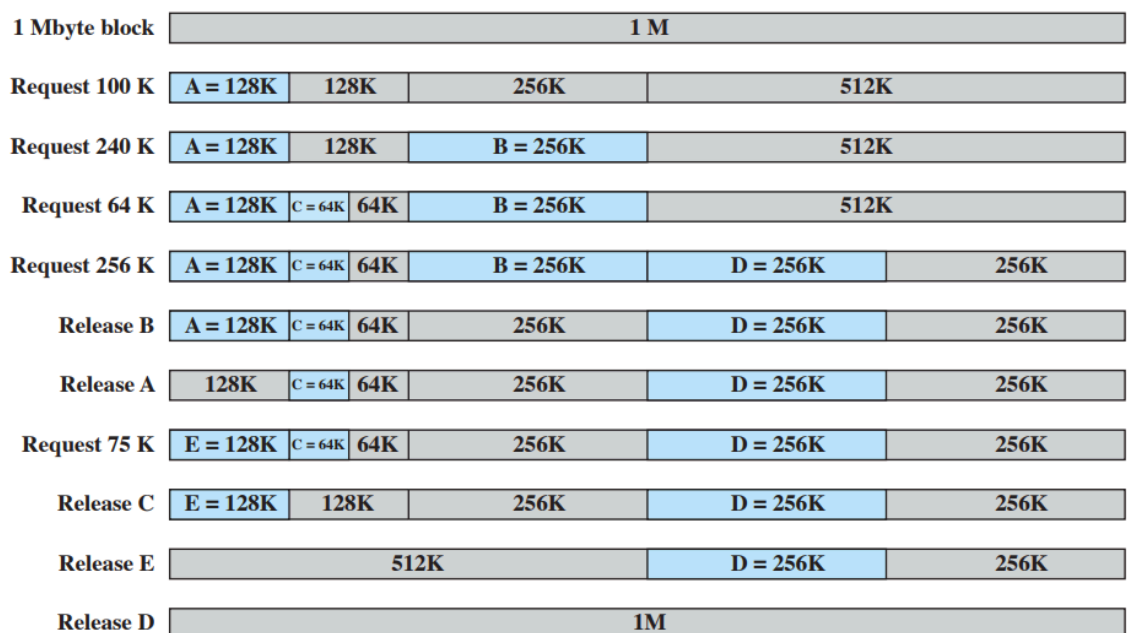
1. Best-fit: sceglie il blocco di dimensione più prossima alla dimensione richiesta
2. First-fit: sceglie il primo blocco di dimensione sufficiente, a partire dall'inizio della memoria
3. Next-fit: sceglie il primo blocco di dimensione sufficiente, a partire dall'ultima partizione creata

C'è infine un compromesso tra la partizione fissa e dinamica.

Questo metodo si chiama Buddy System:

la memoria è vista come un grande blocco di capacità 2^u . Per allocare la memoria, si creano partizioni con dimensione 2^k sufficientemente grande. Per farlo, si spezza il blocco di memoria più grande creando 2 buddies, ripetutamente fino ad arrivare alla dimensione del blocco corretta.

Quando i 2 buddies tornano ad essere liberi, li riuniamo in un unico blocco.



Prima di continuare con gli altri metodi, parliamo di **indirizzi**

- Indirizzo logico: riferimento ad una locazione di memoria indipendente dall'assegnazione della memoria principale al processo. In pratica il processo vede un indirizzo che non corrisponde alla reale posizione in memoria principale.
- Indirizzo fisico: effettiva posizione del dato nella memoria principale.
- indirizzo relativo: è l'indirizzo logico espresso come posizione relativa a un altro indirizzo noto.

Il processo lavora con indirizzi logici/relativi per rimanere portatile e sicuro, senza conoscere la reale disposizione in RAM. Il sistema (loader, kernel, MMU) traduce quegli indirizzi nella corrispondente posizione fisica (indirizzo fisico).

Parliamo anche di **rilocazione**

Il nostro programma viene copiato nella memoria centrale, ma a priori non sappiamo in quale parte venga caricato, quindi nel nostro eseguibile gli indirizzi in cui andare a prelevare le istruzioni e i dati non ci possono stare, saranno indirizzi simbolici (**logici**), una volta caricati, questi indirizzi simbolici devono essere convertiti in indirizzi fisici;

- statica: l'indirizzo fisico viene calcolato una volta, al caricamento (dal loader). Il processo, una volta caricato, non può essere più spostato. Viene caricato sempre nella stessa partizione.
- dinamica: l'indirizzo fisico viene calcolato a runtime. Un processo può occupare differenti partizioni nel corso della sua vita (es. partizioni variabili). Se il processo subisce uno swap da memoria principale a secondaria, quando torna in memoria principale cambia le locazioni di memoria riferite dal programma. Richiede supporto hardware (MMU)

Torniamo alle tecniche:

- **Paginazione**: la memoria principale è partizionata in blocchi di dimensione fissa (relativamente piccola) detti frame.
I processi sono divisi in parti (pagine), che hanno la stessa dimensione dei frame.
Le pagine vengono caricate nei frame, non necessariamente in modo contiguo.
Chiaramente questo risolve il problema della frammentazione ESTERNA! (non interna ma lo minimizza)

0	P1
1	P2
2	P2
3	P1
4	P3

0	Pagina1
1	Pagina2
..	..

Costituzione di
un processo

Costituzione della RAM

Ora abbiamo bisogno di una **tabella delle pagine** (per la traduzione degli indirizzi), che associa ad ogni pagina di un processo un frame.

0	0
1	1
2	2
3	3

Process A
page table

0	—
1	—
2	—

Process B
page table

0	7
1	8
2	9
3	10

Process C
page table

0	4
1	5
2	6
3	11
4	12

Process D
page table

13
14

Free frame
list

Come tradurre l'indirizzo logico a indirizzo fisico nella paginazione?

L'indirizzo logico da 16 bit può essere diviso in due parti:

i primi 6 bit rappresentano il numero della pagina del processo nella tabella delle pagine. Qua troviamo il frame corrispondente

gli altri 10 bit rappresentano l'offset (che sarebbe proprio la posizione variabile desiderata all'interno della pagina a partire dall'inizio della pagina).

L'indirizzo fisico finale è il frame + l'offset (concatenazione)

- Segmentazione

Il programma viene diviso in segmenti anche di lunghezza differente (con un tetto massimo).

L'indirizzo logico è sempre diviso in 2 parti: indice di segmento e offset

Poi carica il programma in memoria con la partizione dinamica (come visto nel partizionamento).

I segmenti possono essere strutturati per riguardare solo codice, dati, stack (quindi divise per contenuto). Sono visibili al programmatore.

Come tradurre l'indirizzo logico a indirizzo fisico nella segmentazione?

i primi n bits riguardano l'indice del segmento (dove troviamo l'indirizzo base)

si verifica che gli altri m bits (offset) non siano più grandi della lunghezza del segmento (si avrebbe un'eccezione di indirizzo)

come nella paginazione l'indirizzo fisico è dato dalla somma dell'indirizzo base + offset (proprio la somma? pare di sì)

Memoria virtuale

Come sappiamo, un processo, per via della rilocalizzazione, o della paginazione o

segmentazione, può trovarsi sia in memoria principale che in memoria secondaria.

Questo non è un problema, basta che la parte del processo che ci interessa (da eseguire) sia nella memoria principale.

- il SO carica in memoria la pagina o segmento
- quando è richiesto un indirizzo non presente in memoria principale si genera un'interruzione
- il SO sospende il processo e il processore è assegnato a un altro processo
- La pagina (o segmento) contenente l'indirizzo logico richiesto è caricata in memoria
- Al termine del caricamento, è generata un'interruzione che fa riattivare il processo al SO

Questa tecnica ha numerosi vantaggi: incremento della multiprogrammazione-> più memoria libera e quindi più processi (solo parte dei processi viene caricata), un processo può essere più lungo della memoria principale.

Quindi, la **memoria virtuale** è la memoria percepita dal programmatore e dall'utente, realizzata dal SO mediante il caricamento dinamico.

Quindi le tecniche di paginazione e segmentazione sono adattabili alla memoria virtuale.

Nel caso della paginazione, non tutte le pagine del processo devono essere nei frame della memoria principale.

Nel caso della frammentazione, non tutti i segmenti del processo devono essere nella memoria principale.

L'hardware deve supportare paginazione e segmentazione, mentre il SO deve gestire il movimento delle pagine o segmenti fra memoria principale e secondaria.

Introduciamo un nuovo termine, **trashing**: stato in cui il sistema spende la maggior parte del suo tempo nel caricamento e scaricamento della memoria, anziché nell'esecuzione dei processi.

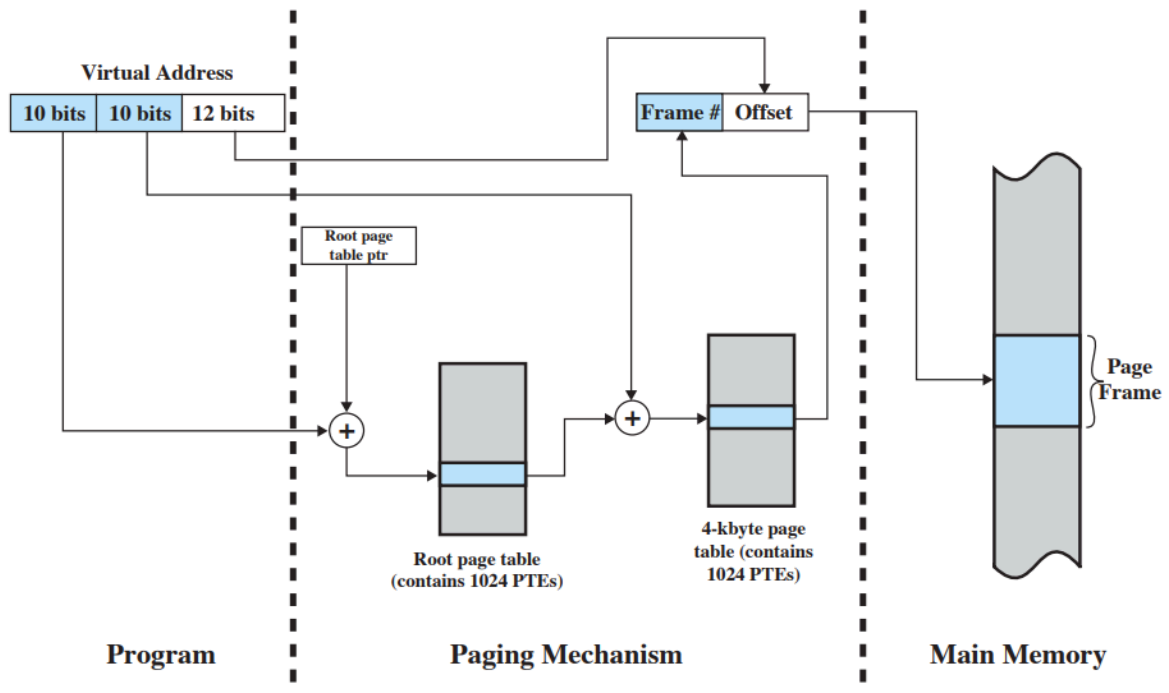
Come possiamo caricare in modo intelligente le pagine in memoria?

Per il principio di località, i riferimenti a dati e codice di un processo che si vanno ad utilizzare tendono a raggrupparsi in poche pagine (o segmenti).

Come funziona la paginazione dinamica?

Nella tabella delle pagine di ogni processo ora c'è anche un bit (P, presenza) che indica se la pagina è caricata in memoria oppure no (il processore guarda come prima cosa questo bit). Poi c'è un bit modificato (dirty bit) che serve ad indicare che tale pagina ha subito delle modifiche da quando è stata caricata in memoria.

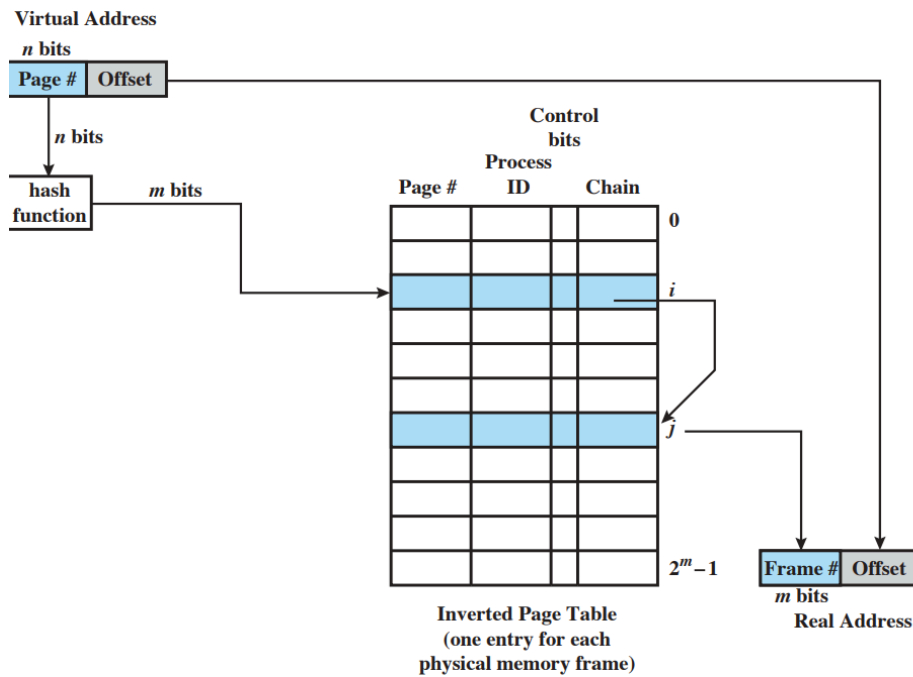
Quando le pagine iniziano ad essere numerose, la tabella delle pagine occupa molto memoria solo per questa (che è uno spreco). La soluzione è la **tabella delle pagine a 2 livelli**. In pratica abbiamo una tabella che mi indica in quale pagina è caricata la tabella delle pagine. L'indirizzo virtuale ora contiene anche dei bit per trovare l'indice della prima tabella, dove troviamo l'indice della pagina contenuta seconda tabella che contiene il frame. Lo svantaggio è che abbiamo un triplo accesso in memoria (evitabile).



In alternativa si usa la tabella delle pagine invertita, dove si utilizza un'unica tabella con tutti i frame della memoria.

Campi della tabella invertita:

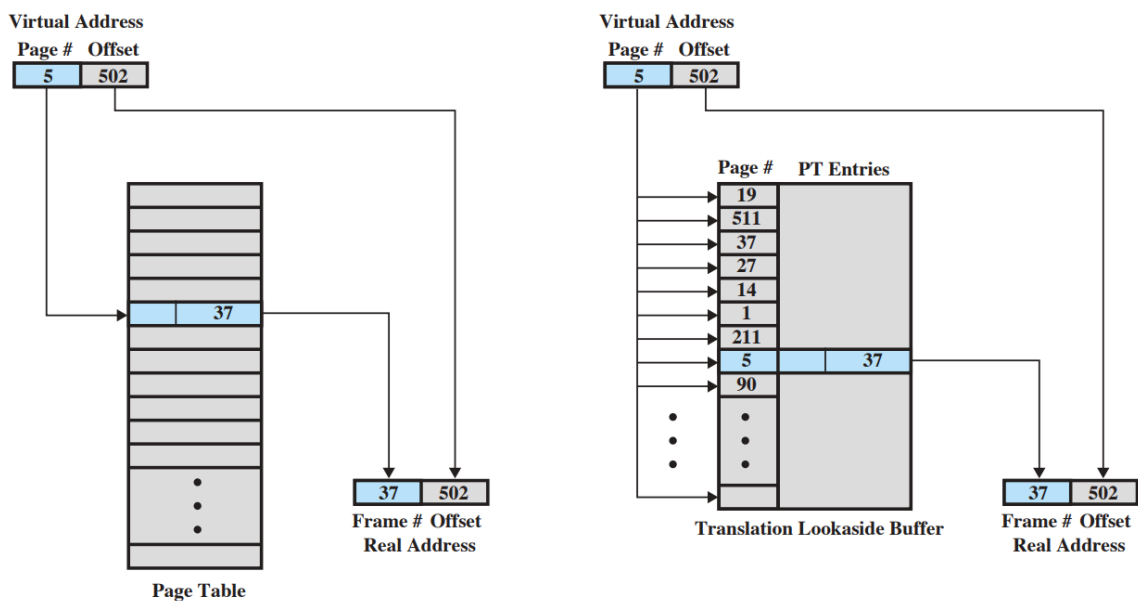
- Numero di pagina;
- Identificatore di processo;
- Bit di controllo;
- Puntatore alla entry successiva.



Altro meccanismo: Translation lookaside buffer

Questo metodo evita il doppio accesso alla memoria della tabella delle pagine (3 nel caso delle tabelle a 2 livelli).

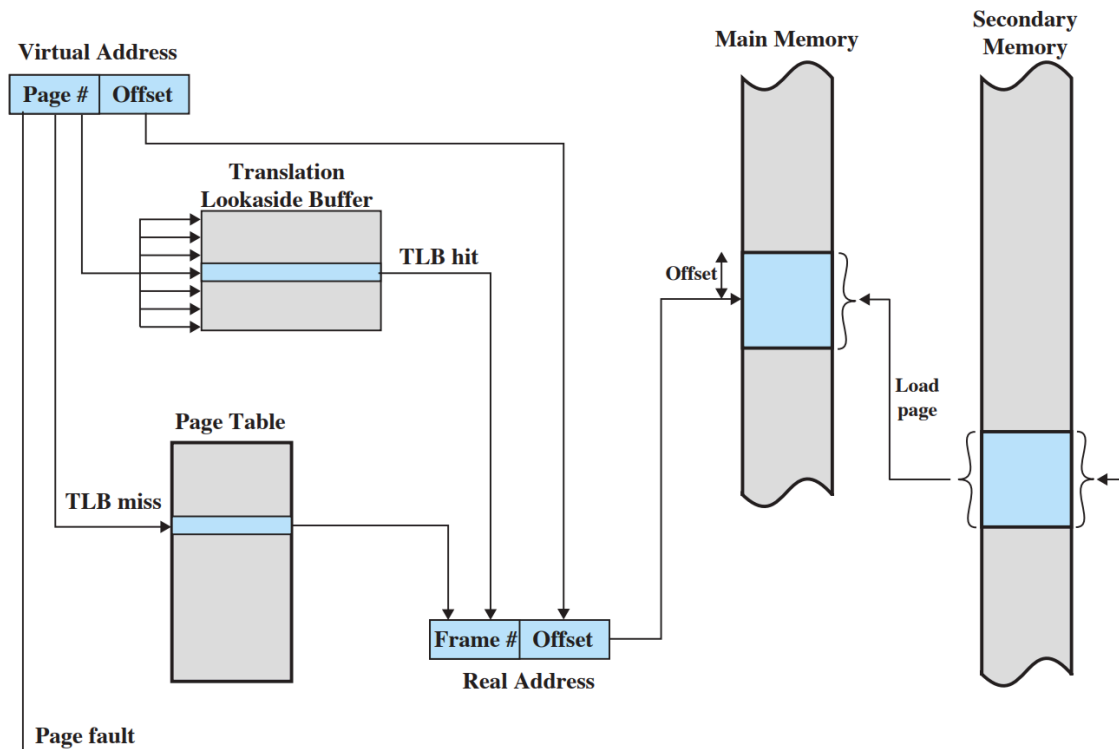
E' infatti una sorta di memoria veloce (cache), contenuta in CPU, contenente le traduzioni usate più di recente (quindi alcune delle tabelle delle pagine) tramite una mappa associativa.



A sinistra c'è il metodo classico, a destra invece abbiamo la mappa del TLB con le traduzioni recenti. Dato un numero di pagina, il TLB controlla tutte le sue entry contemporaneamente per vedere se ha la corrispondenza.

Qua torna il principio di località: le pagine più utilizzate le troviamo nella cache grazie al TLB.

Quando dobbiamo fare una rilocalizzazione(?), verificheremo prima il TLB. Se contiene l'indirizzo (hit) allora abbiamo già il frame della memoria fisica e basta aggiungere l'offset. Se invece non lo contiene (miss), troviamo il frame corrispondente leggendo la tabella delle pagine. Se non troviamo per nulla la pagina, il processo viene interrotto con **page fault**.



Questo è un grosso risparmio di tempo. E' ciò che rende la memoria virtuale performante.

Di che dimensione sono le pagine?

Il problema delle dimensioni delle pagine

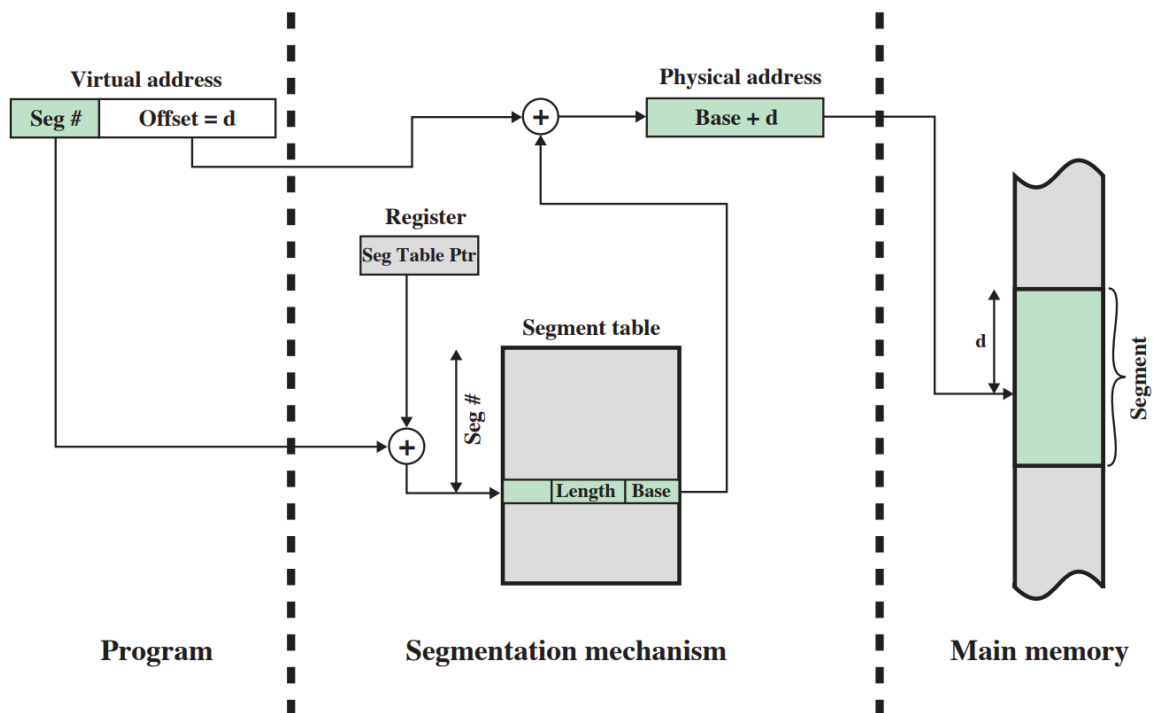
Come scegliamo la dimensione delle pagine? Consideriamo vari fattori:

- pagine piccole riducono la frammentazione interna ma aumentano la dimensione della tabella delle pagine e quindi potrebbe accadere che neanche la tabella delle pagine può stare in memoria principale.
- La scelta è anche condizionata dalle caratteristiche fisiche della memoria secondaria (trasferimento a blocchi)
- Le dimensioni delle pagine influiscono sulla frequenza dei page fault
- Se le pagine sono troppo grandi potrebbero contenere dati che non verranno mai utilizzati e quindi è uno spreco di spazio

Come funziona la segmentazione dinamica?

La novità è che la tabella dei segmenti, oltre a contenere l'indirizzo iniziale (base) del segmento in memoria e la lunghezza del segmento, ora contiene anche un **flag caricato** ed un **flag modificato**

Rilocazione



Se il segmento non è caricato, lo si deve caricare (segment fault, interrompe il programma etc). I problemi sono gli stessi della segmentazione normale, quindi visto che usiamo la partizione dinamica bisogna trovare il blocco di memoria adeguato al segmento da caricare. Poi un problema è la frammentazione esterna.

Come funziona la segmentazione paginata?

Abbiamo infine la segmentazione paginata che prende i vantaggi di entrambi.

Ogni segmento del programma viene diviso in pagine a dimensione fissa, uguali alla lunghezza dei frame.

Infatti la memoria fisica qua è gestita a livello frame.

Vantaggi: elimina la frammentazione esterna

Segmentazione

Visibile al programmatore

Paginazione

Trasparente al programmatore

Virtual Address

Segment Number	Page Number	Offset
----------------	-------------	--------

Segment Table Entry

Control Bits	Length	Segment Base
--------------	--------	--------------

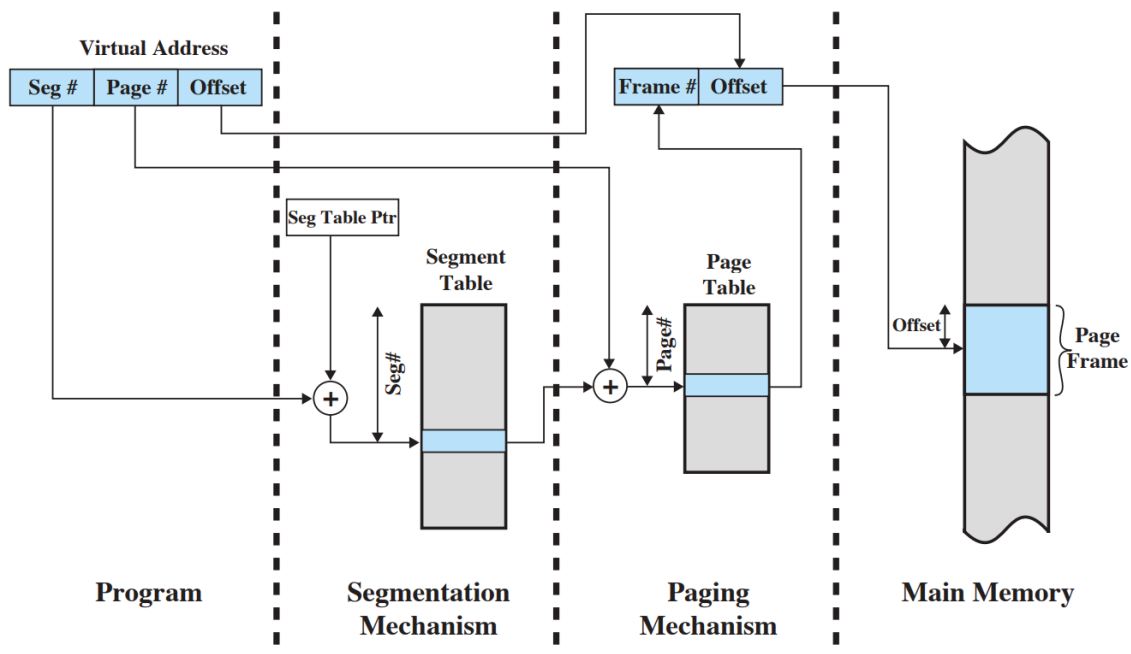
Page Table Entry

P	M	Other Control Bits	Frame Number
---	---	--------------------	--------------

P= present bit

M = Modified bit

Ora abbiamo l'indirizzo virtuale formato da numero di segmento, numero di pagina e offset.



Prima troviamo il segmento nella tabella dei segmenti che ci dà la pagina da guardare nella tabella delle pagine che infine ci dà il frame della memoria.

L'indirizzo fisico è il frame e l'offset.

Politiche di gestione della memoria

Dopo aver deciso se usare o meno memoria virtuale o no, la paginazione, segmentazione o segmentazione paginata, non ci rimane che vedere vari algoritmi per la gestione della memoria.

Prima di tutto ci deve essere una strategia di caricamento: determina quando una pagina deve essere caricata in memoria. Può essere una paginazione a richiesta (la pagina viene

caricata in memoria solo quando c'è un riferimento a un indirizzo della pagina) oppure una prepaginazione

Poi la strategia di posizionamento determina in quale parte della memoria principale la pagina o il segmento va caricato (importante solo per la segmentazione non paginata, può usare i criteri first fit, best fit, next fit).

Strategia di sostituzione

Seleziona la pagina in memoria principale da sostituire quando è necessario caricare una nuova pagina. L'obiettivo è selezionare la pagina che più difficilmente sarà riferita nel futuro prossimo. Più complessa è la politica di sostituzione, maggiore l'overhead per implementarla.

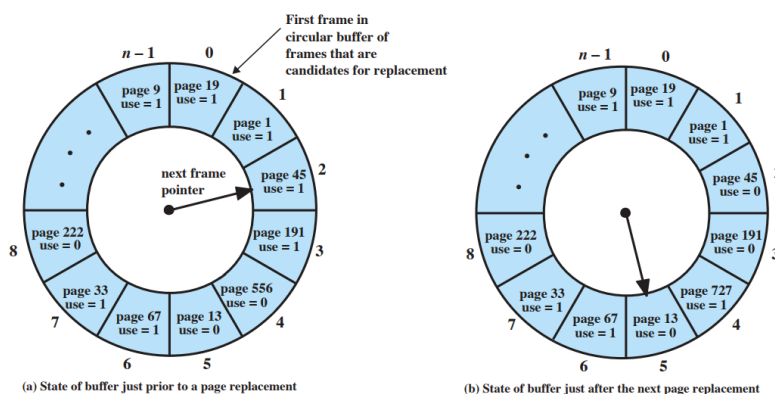
Attenzione! Ci sono però dei frame che non possono essere sostituiti anche se inutilizzati, sono i frame **bloccati (locked)**. Per esempio, il kernel del SO, il buffer I/O sono frame bloccati (gli ultimi solo temporaneamente). Il blocco lo si vede grazie al lock bit vicino al frame (nella tabella delle pagine o dei frame).

Vediamo gli algoritmi:

- Algoritmo ottimale -> sarebbe quello che sostituisce la pagina che in futuro sarà riferita per ultima. E' irrealizzabile ma è utile per i benchmark per confrontarli con altri algoritmi.
- Least Recently Used (LRU) -> sostituisce la pagina che da più tempo non è riferita. Il problema di questo algoritmo è che è difficile da implementare, perché bisogna marcare ogni pagina col tempo dell'ultimo riferimento e questo comporta un grande overhead.
- First-in-First-out (FIFO) -> utilizza un buffer circolare e sostituisce la pagina che è stata per più tempo in memoria. Facile da implementare.
- Strategia dell'orologio -> è uno dei metodi più efficienti, abbiamo un flag associato a ogni frame (bit di utilizzo) che viene settato a 1 quando la pagina è caricata o riferita.

L'algoritmo scandisce circolarmente i frame:

1. se il bit di utilizzo è 1, viene settato a 0.
 2. se il bit di utilizzo è 0, viene selezionato per la sostituzione.
- dando così il tempo alle pagine di essere utilizzate.



Una versione modificata dell'algoritmo dell'orologio utilizza sia il bit di utilizzo che il bit di modifica.

Buffer per pagine (fa sempre parte della strategie di sostituzione)

Migliora la gestione della memoria virtuale. L'idea è quella di utilizzare un buffer per le **pagine scaricate** formato da due liste: pagine libere e pagine modificate.. E' utile perché mentre queste pagine sono in queste liste possono essere recuperate senza accedere al disco, migliorando le prestazioni

Gestione del residence set

Dimensione del residence set

Il SO decide quante pagine per ogni processo tenere in memoria principale (**residence set**).

Sarebbe utile decidere a priori per ogni processo quante pagine tenere in memoria.

Come decidiamo la dimensione del Residence set?

Ci sono 2 possibilità:

1. Allocazione fissa: il processo ha un numero fisso di frame in cui essere eseguito, a ogni errore di pagina si scarica una pagina del processo che lo ha determinato.
2. Allocazione variabile: il numero dei frame allocati a un processo può variare nel tempo.

Ambito di sostituzione

Ambito locale -> la pagina da sostituire fa parte del processo che ha determinato l'errore di pagina

Ambito globale -> considerata tutte le pagine (non bloccate) nella memoria.

Allocazione variabile, ambito locale -> ad un processo caricato in memoria gli vengono allocati un certo numero di frame, se c'è un errore, il numero di frame aumenta (si rivaluta periodicamente il numero)

Allocazione variabile, ambito globale -> Il SO ha una lista di frame liberi che in caso di errori di pagina vengono messi nel residence set. In assenza di frame liberi, una pagina di qualunque processo può essere scaricata.

Working set

E' l'insieme delle pagine a cui un processo ha fatto riferimento durante un intervallo di tempo virtuale, ovvero un intervallo espresso non in un tempo reale ma in numero di operazioni di accesso alla memoria.

Una strategia potrebbe essere quella di rimuovere dal residence set le pagine che non sono nel working set.

Strategia di cleaning

Su richiesta -> la pagina è salvata in memoria secondaria solo quando deve essere sostituita

Precleaning -> permette il salvataggio di pagine a lotti.

Controllo del caricamento

Controlla il grado di multiprogrammazione. Se il sistema rileva che sta entrando in trashing, deve ridurre il grado.

Gestione dell'I/O

Abbiamo varie categorie di dispositivi I/O

- Leggibili dall'uomo (per l'interazione con l'utente, come tastiera, mouse, stampanti)
- Leggibili dalla macchina (per la comunicazione con componenti elettronici (chiavette USB, dischi, sensori))
- Dispositivi di comunicazione (per comunicare con dispositivi remoti, come modem e scheda di rete)

Questi dispositivi si differenziano per varie caratteristiche, come la velocità di trasferimento, le applicazioni, tipi di errori...

Gestione dell'I/O del processore

Ci sono 3 tecniche:

1. I/O programmato -> il processore genera il comando I/O per conto di un processo. Molto inefficiente perché la CPU aspetta che l'operazione sia completata
2. I/O guidato dalle interruzioni -> il processore genera il comando I/O e poi va a fare altro. Poi il processo manda l'interruzione quando ha fatto. Intanto il processo può essere bloccante (aspetta) o non bloccante (continua l'esecuzione)
3. DMA (Direct Memory Access) -> un modulo hardware che controlla lo scambio di dati tra memoria principale e dispositivo I/O

Via via, il processore è meno coinvolto nella gestione dell'I/O.

Obiettivi di progettazione del Software I/O

Efficienza perché le operazioni di I/O sono chiaramente un collo di bottiglia e molto lente rispetto alla memoria principale e processore

Generalità perché vanno supportati vari dispositivi differenti tra loro fornendo un'interfaccia uniforme.

Buffering

Il buffering è una tecnica che migliora le prestazioni evitando overhead che consiste nel trasferire input in anticipo sulla richiesta e trasferire output in ritardo rispetto alla richiesta.

- Nei dispositivi a blocchi (dischi e chiavi USB) si possono trasferire blocchi di dimensione fissa
- Nei dispositivi a flusso (tastiera, stampanti, mouse) si trasferiscono flussi di byte

Strategie di buffering

- Buffer singolo nei dispositivi a blocchi: sistema più semplice. Quando un processo esegue una richiesta di I/O, il SO assegna un unico buffer all'operazione dove viene caricato l'input dal dispositivo per copiarlo nello spazio utente. Mentre il processo elabora un blocco di dati, già viene richiesto il blocco successivo (input anticipato)
- Buffering singolo nei dispositivi a flusso: una linea alla volta o un byte alla volta
- Doppio buffer: usa 2 buffer. Mentre il processo consuma i dati da un buffer, si riempie l'altro così da non aspettare che il primo buffer sia completamente vuoto.
- Buffer circolare: Più di 2 buffer caricati e scaricati circolarmente.

Tempo di accesso

Il disco è diviso in tracce che hanno diversi settori. La testina scorre sulle tracce

Il tempo totale per accedere a un dato su disco (tempo di accesso) è la somma di:

tempo di ricerca -> tempo per muovere la testina sulla traccia giusta

ritardo rotazionale -> tempo per attendere che il settore giusto passi sotto la testina
tempo di trasferimento -> tempo per leggere/scrivere i dati effettivi

Se si devono leggere tracce adiacenti, il tempo di ricerca per le letture successive è quasi a 0.

Quali sono le politiche per la schedulazione delle richieste al disco?

Dato che il tempo di ricerca è la componente dominante del tempo di accesso, l'obiettivo delle politiche di schedulazione è ridurre il tempo medio di ricerca.

- FIFO: richieste processate in ordine di arrivo, approssima la schedulazione random
- Priorità: non ottimizza l'uso del disco. Da priorità a lavori brevi e ai processi interattivi. Chiaramente la politica è gestita fuori dal software del disco. Un problema potrebbe essere la starvation dei lavori a bassa priorità
- SSTF (Shortest seek time first): Seleziona la richiesta più vicina alla posizione attuale della testina. Può causare **starvation** per le richieste lontane
- SCAN: detto anche algoritmo dell'ascensore. La testina si muove in un'unica direzione e soddisfa tutte le richieste su ciascuna traccia finché non raggiunge l'ultima traccia, poi inverte la direzione. Ha buone performance e prima o poi tutte le richieste vengono soddisfatte ma favorisce i settori centrali chiaramente.
- C-SCAN (Circular SCAN): come SCAN, quando arriva all'ultima traccia riparte dalla prima, quindi mantiene la stessa direzione
- N-Step-SCAN: Le richieste sono inserite in sotto-coda di lunghezza N che vengono processate con SCAN. Le nuove richieste durante una scansione vanno in una nuova coda
- FSCAN: 2 sottocoda, si eseguono le richieste di una coda e le richieste ricevute durante la scansione vanno nell'altra coda

Gestione dei file

Un file è una collezione di dati creata dall'utilizzatore.

Proprietà:

1. Persistenti (se tutto funziona come dovrebbe, il file rimane là)
2. Condivisione (hanno un nome e dei permessi di accesso che ne permettono la condivisione)
3. Struttura (sono organizzati in strutture gerarchiche).

File System

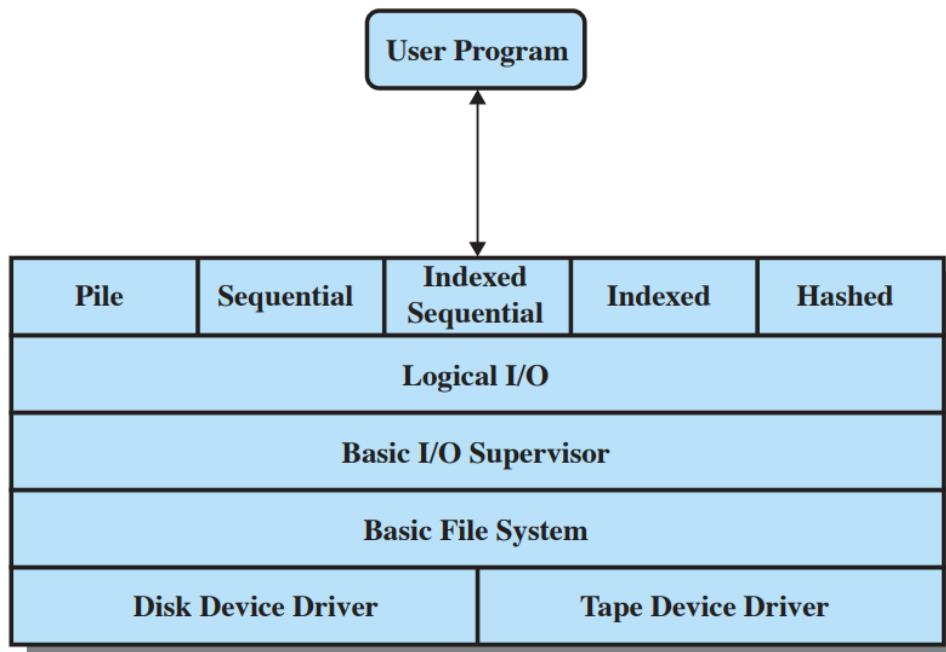
È il modulo del SO che si occupa di tutto ciò che riguarda i file. Fornisce gli strumenti per operare sui file, come crea, cancella, apre, chiude, legge, scrive.

Allo stesso tempo deve garantire la validità dei dati e ottimizzare le prestazioni. Deve anche prevedere il supporto I/O per i vari tipi di dispositivi di memorizzazione (disco, usb...).

Devono anche fornire un'interfaccia standardizzata.

per garantire tutto ciò, il file system utilizza una struttura a strati:

Architettura del sistema di gestione dei file



- Driver dei Dispositivi (Livello più basso): Comunica direttamente con l'hardware (es. controller del disco). Avvia le operazioni I/O e gestisce il loro completamento.
- Basic File System (Base del file system): Si occupa dei blocchi fisici di dati. Gestisce lo scambio di blocchi con il disco e organizza i buffer in memoria. Non conosce la struttura logica del file.
- Supervisore dell'I/O base: Gestisce l'inizio e la fine delle operazioni I/O sui file. Si occupa della schedulazione del disco per ottimizzare le prestazioni e alloca i buffer.
- I/O logico: Abilita l'accesso ai file a livello di record logici, nascondendo la struttura a blocchi.
- Metodi di accesso (Livello più alto): Fornisce l'interfaccia standard alle applicazioni. I diversi metodi riflettono le diverse organizzazioni dei file (es. sequenziale, indicizzato).

Directory

E' una struttura che contiene informazioni sui file.

Attributi -> Sono i metadati che descrivono un file, come:

- Basic Information: Nome, tipo, organizzazione.
- Address Information: Dove si trova fisicamente sul disco.
- Access Control Information: Chi può fare cosa (proprietario, permessi).
- Usage Information: Date di creazione/modifica, chi lo sta usando, ecc.

La directory può essere strutturata così:

1. Schema a due livelli -> si ha una directory principale e una directory per ogni utente
2. Schema ad albero -> ogni directory contiene sottodirectory e file

Come sono organizzati i record all'interno di un file?

Ci sono vari meccanismi:

- File a mucchio: i dati vengono registrati in ordini di arrivo, ogni nuovo record viene messo in coda al file.
- File sequenziale: i record hanno un formato fisso e vengono ordinati secondo un campo chiave.
- File sequenziale indicizzato: aggiunge un indice ai per permettere un accesso rapido ma random.
- File indicizzato: record accessibili attraverso gli indici. Accesso veloce.
- File diretto o a hash: accesso velocissimo. Grazie alla funzione hash con input la chiave del record si calcola direttamente l'indirizzo del blocco fisico.

Come vengono allocati i file nel disco fisico?

Prima di tutto, in memoria secondaria, un file consiste in una collezione di blocchi.

E' il SO a gestire l'allocazione dei blocchi ai file. L'approccio scelto ha delle conseguenze sulla gestione dello spazio libero.

La struttura che tiene traccia dei blocchi allocati ai file si chiama FAT.

La dimensione del file è determinata dalla scelta tra la preallocazione o l'allocazione dinamica.

Nella preallocazione si decide la dimensione del file al momento della creazione (difficile da calcolare).

Nell'allocazione dinamica si aumenta la dimensione al bisogno.

Poi l'allocazione può essere contigua, a catena (concatenata) o indicizzata.

1. **contigua:** blocchi posizionati in modo adiacente sul disco. Potremmo dover ricorrere alla compattazione per avere dei blocchi contigui liberi.
2. **a catena:** ogni blocco contiene un puntatore al blocco successivo. E' più lenta
3. **indicizzata:** il primo blocco contiene tutti i puntatori verso gli altri blocchi. E' molto rapido in casi in cui bisogna accedere subito all'ultimo blocco.

Gestione dello spazio libero

E' chiaramente necessario che il file system sia a conoscenza dei blocchi ancora disponibili.

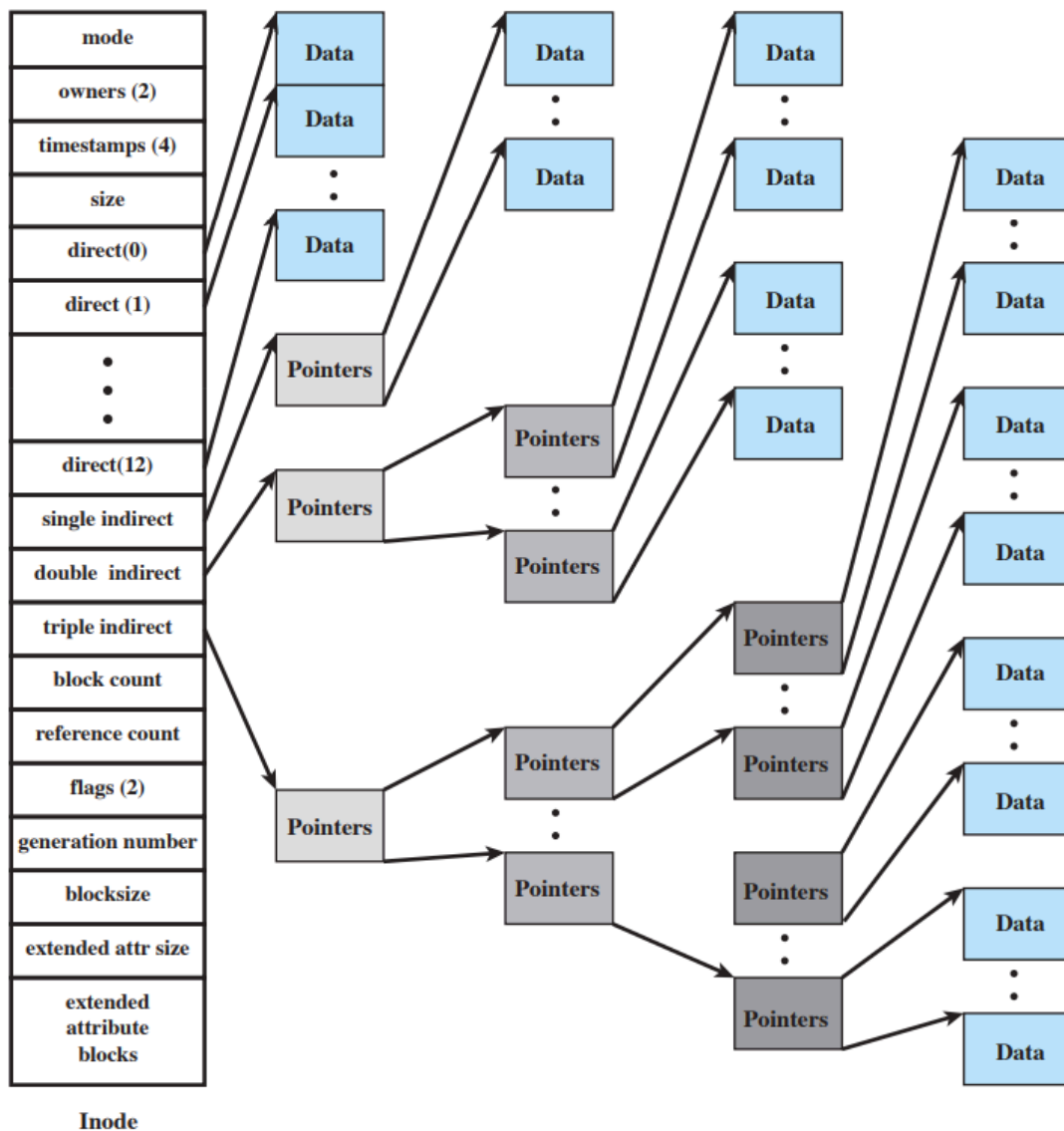
Questo è possibile grazie alla tabella di allocazione del disco, realizzabile in vari modi.

- bitmap: un vettore di bit per ogni blocco. 0 = libero, 1 = allocato
- porzioni libere concatenate: i blocchi liberi sono collegati in una lista concatenata
- indicizzazione: lo spazio libero è trattato come un file con un indice dei blocchi liberi

Come funzionano realmente le cose in UNIX?

Sostanzialmente tutto è un file. Le directory sono file, i link simbolici, pipe.

C'è una struttura dati centrale chiamata i-node (index node) che contiene tutte le informazioni sui file necessarie. Ogni file ha il suo i-node, ma più file possono essere associati allo stesso i-node.



Contiene puntatori diretti (per i file piccoli), e puntatori indiretti (singoli, doppi, tripli) che puntano a blocchi di puntatori, permettendo di gestire file di dimensioni enormi.

Per finire, il file system UNIX riesce su un singolo disco logico.

Un disco UNIX è organizzato in: Boot block, Superblock (contiene info sul file system), Tabella degli i-node, e Blocchi dati.

Perfetto, hai colto i vantaggi principali. Ora mettiamo in relazione la tua risposta con i termini esatti usati nella slide 15 per renderla completa.

Analisi della tua risposta:

- Hai spiegato correttamente il principio base: il kernel si limita alle funzioni essenziali ("Tutto il possibile trasferito a programmi livello-utente").
- Hai identificato correttamente i vantaggi: facilità di aggiornamento e maggiore affidabilità ("semplicità di aggiornamento e debugging", "affidabilità").

La slide 15, però, menziona anche il meccanismo di comunicazione e uno svantaggio specifico.

Una risposta completa basata sulla slide 15 sarebbe:

Il principio fondamentale dell'architettura a microkernel è quello di **spostare quante più funzioni possibili dal kernel a programmi a livello utente**, chiamati **server**. Il kernel mantiene solo le funzioni più basilari (come gestione della comunicazione e schedulazione a basso livello).

La comunicazione tra i componenti (client e server) avviene tramite **scambio di messaggi**.

- **Vantaggi principali:**
 - **Semplicità di aggiornamento e debugging:** Si può modificare un server senza dover ricompilare l'intero kernel.
 - **Portabilità e affidabilità:** Un errore in un server non fa crashare l'intero sistema, ma solo quel servizio.
-
- **Svantaggio principale:**
 - **Overhead:** La comunicazione tramite scambio di messaggi tra processi a livello utente è più lenta e introduce un maggiore overhead rispetto a chiamate di funzione dirette all'interno di un kernel monolitico.
-

La tua risposta era ottima, mancava solo la menzione dell'overhead come svantaggio esplicito indicato dalla slide.

Prossima domanda:

Nella presentazione "Gestione della memoria", la slide 17 introduce il **Buddy System**. Questa tecnica è descritta come un "compromesso tra partizioni fisse e dinamiche".

Spiega brevemente come funziona il meccanismo di **allocazione e rilascio** nel Buddy System. In particolare, cosa succede quando si deve allocare un blocco di una certa dimensione e non è disponibile un blocco di quella dimensione esatta? E cosa succede quando due blocchi "buddies" vengono liberati?

